

“Federated-Deep Reinforcement Learning for Spectrum Sensing in Cognitive IOT Networks”

Major Project Report

*Submitted in Partial Fulfillment of the
Requirements for the Degree of*

BACHELOR OF TECHNOLOGY

IN

INFORMATION AND COMMUNICATION TECHNOLOGY (ICT)

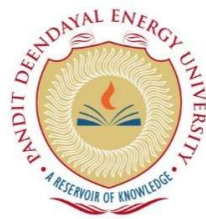
By

Chirag Chauhan and Meet Raithatha

22BIT186 and 22BIT006

Under the Guidance of

Dr. Abhishek Joshi



**Department of Electronics and Communication Engineering,
School of Technology, Pandit Deendayal Energy University,
Gandhinagar 382 426**

May 2026

Certificate of Originality of Work

I hereby declare that the B.Tech. Project entitled “Federated – Deep Reinforcement Learning for Spectrum Sensing in Cognitive IOT Network” submitted by us for the partial fulfillment of the degree of Bachelor of Technology to the Dept. of information and communication Technology at the School of Technology, Pandit Deendayal Energy University, Gandhinagar, is the original record of the project work carried out by me under the supervision of Dr. Abhishek Joshi.

I also declare that this written submission adheres to University guidelines for its originality, and proper citations and references have been included wherever required.

I also declare that I have maintained high academic honesty and integrity and have not falsified any data in my submission.

I also understand that violation of any guidelines in this regard will attract disciplinary action by the institute.

Name of the Student: Chirag Chauhan

Roll Number of the Student: 22BIT186

Name of the Student: Meet Raithatha

Roll Number if the Student: 22BIT006

Signature of the Student:

Name of the Supervisor: Dr. Vivek Pandit

Designation of the Supervisor:

Signature of the Supervisor:

Place:

Date:

Certificate from the Project

Supervisor/Head

This is to certify that the Major Project Report entitled “Federated – Deep Reinforcement Learning for Spectrum Sensing in Cognitive IOT Network” submitted by Mr. Chirag Chauhan and Mr. Meet Raithatha, Roll No. 22BIT186, and Roll No. 22BIT006 towards the partial fulfillment of the requirements for the award of degree in Bachelor of Technology in the field of Information and Communication Technology Engineering from the School of Technology, Pandit Deendayal Energy University, Gandhinagar is the record of work carried out by her under our supervision and guidance. The work submitted by the student has in our opinion reached a level required for being accepted for examination. The results embodied in this comprehensive project work to the best of our knowledge have not been submitted to any other University or Institution for the award of any degree or diploma.

Dr. Abhishek Joshi

Faculty Mentor

Dr. Paawan Sharma

HOD ICT

Dr. Anirbid Sircar

Director

Place

Date

Acknowledgement

This project would not have been possible without the support and guidance of many people who encouraged me throughout this journey. I would like to sincerely thank my faculty mentor for their constant guidance, valuable suggestions, and encouragement at every stage of the project. Their support helped me stay motivated and continuously improve my work and understanding.

I am also grateful to my industry mentor for sharing practical insights and real-world knowledge that added great value to this project. Their guidance helped me understand how academic concepts can be applied to real business challenges in the retail and e-commerce domain.

I would also like to thank the Department of Electronics and Communication Engineering, School of Technology, Pandit Deendayal Energy University for providing a supportive learning environment and the necessary resources to successfully complete this project.

A special thanks to my friends and classmates for their support, discussions, and encouragement throughout this journey. Working and learning alongside them made this experience more enjoyable and meaningful.

Finally, I am deeply thankful to my family for their unconditional support, patience, and belief in me. Their encouragement gave me the strength and confidence to complete this work successfully.

Chirag Chauhan and Meet Raithatha

Abstract

Effective spectrum management becomes highly necessary due to the widespread deployment of wireless communication networks and IoT devices. The issue of static spectrum allocation causes inefficiency, interference problems, and spectrum shortage. In this regard, CRNs prove to be an effective technology by allowing Secondary Users (SUs) to utilize spare spectrum without causing disturbance to Primary Users (PUs). Yet, existing techniques of spectrum sensing have several drawbacks such as inaccurate estimation, lack of adaptivity under low SNR, and large communication overheads in federated IoT networks.

To overcome these limitations, this work presents a Federated Deep Reinforcement Learning-based Spectrum Sensing Architecture in Cognitive IoT Network on top of a hierarchical federated learning paradigm. IoT devices (Raspberry Pi and ESP32) conduct intelligent spectrum sensing and adaptive channel selection through collaboration and maintain local data privacy, thus sharing only the parameters of models but not data itself. Our framework involves a BiLSTM-based spectrum sensing and modulation classifier using RadioML 2018.01A signal data along with Dueling DQN algorithm used in conjunction with Multi-Head Self-Attention technique.

The system operates across a three-layer HFL architecture comprising IoT clients, Edge Servers, and a Central Aggregation Server, with Federated Averaging (FedAvg) enabling distributed collaborative learning. This integrated design simultaneously improves sensing accuracy, reduces communication overhead, ensures data privacy, and optimizes spectrum utilization. By unifying Deep Learning, Reinforcement Learning, and Federated Learning, the proposed framework delivers an intelligent, scalable, and privacy-preserving solution for dynamic spectrum management in next-generation Cognitive IoT Networks.

LIST OF FIGURES

FIGURE No.	DESCRIPTION	PAGE
1	Star-Mesh Hybrid Network Topology	24
2	Constellation Diagram	30
3	I/Q Phase Graph Of (X,Y) Sample	31
4	Constellation Diagram For QPSK	31
5	Data Pipeline Flow Chart	48
6	Training Pipeline Flowchart	49
7	Evaluation Pipeline Flowchart	50
8	Full System Topology	50
9	Federated Learning Communication Links And Flowchart	51
10	Central-Level Global LSTM Accuracy, Loss, DQN Reward, Collision Rate	55
11	Edge Server DQN Reward Comparison Between Two Edge Device	57
12	Edge Device 1 And 2 LSTM Accuracy, LSTM Loss, DQN Reward, Collision Rate	58
13	IOT Client 4 LSTM Accuracy, LSTM Loss, DQN Reward, Collision Rate	60
14	IOT Client 3 LSTM Accuracy, LSTM Loss, DQN Reward, Collision Rate	61
15	IOT Client 2 LSTM Accuracy, LSTM Loss, DQN Reward, Collision Rate	62
16	IOT Client 1 LSTM Accuracy, LSTM Loss, DQN Reward, Collision Rate	63

LIST OF TABLES

TABLE NO.	DESCRIPTION	PAGE
1	COMMUNICATION PROTOCOL TABLE SHOWING HOW THE DEVICES ARE CONNECTED TO EACH OTHER	24
2	DETAIL OF RADIOML 2018 DATASET	29
3	LSTM LAYER CONFIGURATION WITH PARAMETER AND VALUES	38
4	CONVOLUTION 1D LAYER WITH INPUT AND OUTPUT CHANNELS VALUES	39
5	COMBINE MODEL WEIGHTS ASSIGNED	41
6	EPSILON GREEDY CONFIGURATION TABLE	44
7	ASSIGN THE IP ADDRESS	46
8	SOFTWARE REQUIREMENTS	51
9	LIBRARIES FOR PYTHON PROGRAMMING	52
10	ACCURACY FOR EACH 20 ROUNDS, START, END, AND PEAK	54
11	IOT CLIENTS FINAL ACCURACY, PEAK ACCURACY, FINAL REWARD AND COLLISION RATE PERCENTAGE	59

Chapter 1 Introduction

1.1 Background and Prologue

With the advent of wireless communication network and Internet of Things (IoT) devices, the requirement for effective use of radio spectrum resources is high. The conventional fixed-spectrum allocation approach tends to cause the spectrum band allocated by the license underutilization problem, so that the spectrum can be generated in a more congested network [1]. To make the best use of the unused bands of spectrum, Cognitive Radio Networks (CRNs) were introduced, which allows the Secondary Users (SUs) to access the bands and use the spectrum for their purpose [2].

One of the fundamental operations CPU implemented in Cognitive Radio systems is a "Spectrum sensing" operation. It enables IoT devices to discover spectrum gaps dynamically prior transmission. Various spectrum sensing methods like energy detection, matched filter, cyclo-stationary feature detection have been extensively adopted in the CRNs [3, 4]. The methods have some drawbacks, such as:

- Low detection accuracy at low SNR conditions
- High computational complexity
- Sensitivity to noise uncertainty
- Limited adaptability in dynamic wireless environments

There has been a recent improvement in the performance of the spectrum sensing systems using the help of the developments of Artificial Intelligence (AI) and Machine Learning (ML). DEEP LEARNING models can automatically obtain significant spectral characteristics from the original signals data, and improve the accuracy of sensing based on conventional methods. In addition, Reinforcement Learning (RL) can help an intelligent agent adaptively learn the best spectrum access policy as they interact with the wireless environment [6].

Centralized AI-based systems yields better performance but must transmit a lot of sensing data from the individual sensors to a central server, resulting in high communication overhead and privacy issues. Federated Learning (FL) is an alternative approach that solves these challenges by enabling these distributed edge devices to jointly train a global model without embedding any locally stored data [7].

There is a need for spectrum, which is a scarce natural resource whose demand is growing rapidly due to wireless technology proliferation and expansion of Internet of Things (IoT). Various regulatory studies have indicated that several licensed bands tend to stay underutilized for long durations, whereas unlicensed bands get extremely congested. The advent of Cognitive Radio Networks (CRNs) has come forward as a viable alternative since it permits SUs to exploit unoccupied bandwidths without affecting licensed Primary Users (PUs) operations. One of the primary responsibilities of every CRN lies in its ability to perform spectrum sensing to determine whether PU exists or not in any specific frequency band prior to transmission.

Nonetheless, existing sensing methodologies like energy detection, matched filtering, and cyclostationary feature detection lack practicality due to their dependence on high noise uncertainty, complex computations, and poor results obtained in a dynamic environment. In this context, our proposed research attempts to overcome the existing deficiencies by integrating Federated Learning (FL) with Deep Reinforcement Learning (DRL) to design a new paradigm of intelligent decentralized spectrum sensing for Cognitive IoT Networks. Our federated DRL model provides an advantage over other approaches as it uses distributed learning at the edge without transmitting raw data to a central server.

The first requirement is that there is a need for spectrum. This is a rare natural resource, and its demand is increasing quickly because of the widespread adoption of wireless technologies and the growth of the Internet of Things (IoT). Recent advancements in Artificial Intelligence, Deep Learning, and Reinforcement Learning have made intelligent spectrum sensing possible. This process involves automatically extracting intricate spectral characteristics from raw I/Q data and learning how to optimally access the spectrum through adaptive approaches. The major problem with such learning models is that they are designed to work in centralized architectures, which means that IoT devices

must send their raw sensing data to a central server. This poses a serious challenge for privacy, communication, scalability, and latency. To overcome this issue, we propose a Federated Deep Reinforcement Learning (FDRL) model that will allow IoT devices to collaborate in training a global model while maintaining their local data.

1.2 Motivation

An ever growing number of IoT applications and wireless technologies have pushed the available spectrum resources to the breaking point. Static spectrum allocation methods can't efficiently use the bandwidth, leading to spectrum shortage and network congestion. Concurrently, numerous bands are not used during any particular period, but utilized at other times [2].

Moreover, typical spectrum sensing algorithms also have some limitations in dynamic wireless channels. These systems are typically less accurate in sensing in low Signal-to-Noise Ratio (SNR) and dynamic channel environments.

- Cognitive IoT Networks can better serve spectrum utilization in terms of efficiency.
- To minimize conflicts with the Primary Users (PUs)
- To improve the sensing accuracy by intelligent learning models.
- To minimize communications overhead in distributed networks.
- To preserve privacy of sensing data using Federated Learning

The framework would be scalable and adaptable to develop a spectrum sensing framework with flexibility and scalability in mind.

Deep Reinforcement learning empowers adaptive rule learning mechanisms and Federated Learning/If allows collaborative decentralized learning. This represents an interesting combination for better performance in an intelligent spectrum sensing system.

1.3 Problem Statement

With the proliferation of devices, such as Internet of Things (IoT), wireless sensor networks and next generation communication systems, the amount of wireless spectrum usage has surged significantly. Moreover, the demand for reliable and fast wireless communication is constantly present for modern applications, such as smart cities, autonomous systems, industrial automation, and intelligent healthcare.

The radio frequency spectrum is, however, a resource that is finite and the current static spectrum allocation paradigm has resulted in inefficient use of the radio frequency spectrum with numerous licensed bands underutilized and others severely congested [1, 2].

An increased number of Internet of Things (IoT) devices, wireless sensor network, and next generation wireless communications have brought about an increase in the demand for wireless spectrum. With today's demands for reliable and high throughput wireless connections for applications such as smart cities, automation, industrial and intelligent healthcare, among others, radio frequency spectrum becomes a scarce commodity due to static spectrum allocation policy which leads to some spectrum bands being heavily used while other spectrum remain unutilized [1, 2]. Cognitive Radio Networks (CRN), thus, provide a means for efficient utilization of radio spectrum by allowing secondary users' access to idle spectrums.

However, successful implementation of cognitive radio networks depends on accurate sensing of the spectrum which turns out to be very difficult in practice since there exist numerous challenges in actual spectrum sensing such as wireless channel fading effect, shadowing effect [3, 4], hidden node problem [5], noise variability, and fast changing channels. Current sensing methods, however, are plagued with their respective demerits. Energy detection technique is not efficient under conditions of low Signal-to-noise ratio while the matched filter is dependent on the known signal from the primary user.

The potential of Wireless Communications Systems has been developed by recent developments in Artificial Intelligence (AI) and Deep Learning (DL) which make it

possible to extract intelligent features and make adaptable decisions in wireless communications system [5]. In the same way, the Reinforcement Learning (RL) method has demonstrated great promise for addressing dynamic spectrum access and channel selection issues. Most current AI-empowered spectrum sensing systems, however, are built with a centralized learning approach where distributing sensors of the IoT devices receive their signals and then transfer them to a central unit to build a learning model. These techniques have heavy communication overhead, scaling issues, latency and privacy concerns of having raw sense data exposed [6], [7].

The major problems identified in existing spectrum sensing systems are summarized as follows:

1. Inefficient utilization of available radio spectrum resources.
2. Low sensing accuracy in noisy and dynamic wireless environments.
3. High computational complexity of advanced sensing techniques.
4. Communication overhead caused by centralized training architectures.
5. Privacy concerns due to sharing of raw sensing data.
6. Limited scalability for large-scale Cognitive IoT Networks.
7. Increased interference probability with Primary Users (PUs).

Hence, an intelligent, decentralized and privacy-preserving spectrum sensing framework that can adapt to the dynamic wireless environment and enhance the spectrum sensing accuracy and utilization efficiency is needed. To resolve these issues, this work suggests adopting an adaptive spectrum access decision strategy through Federated Deep Reinforcement Learning that does not share raw local data for collaborative model training. To solve these problems, an adaptive spectrum access decision strategy based on Federated Deep Reinforcement learning is proposed in this work, by which the raw local data is not shared and DPRL collaboratively trains the model.

1.4 Objectives

The main aim of this project is to create an intelligent spectrum sensing algorithm using Federated Deep Reinforcement Learning for Cognitive IoT networks.

Project sub-objectives are as follows:

1. To study the architecture and working of Cognitive Radio Networks.
2. To analyze existing spectrum sensing techniques and their limitations.
3. To implement Deep Reinforcement Learning for intelligent channel selection.
4. To integrate Federated Learning for decentralized collaborative model training.
5. To improve sensing accuracy and spectrum utilization efficiency.
6. To minimize communication overhead and preserve user privacy.
7. To evaluate system performance using various performance metrics.

1.5 Scope of the Project

The objective of this project is to design an intelligent spectrum sensing approach for Cognitive IoT networks by implementing Federated Deep Reinforcement Learning (FDRL), which allows secondary users to detect and utilize any available spectrum band without affecting the primary users. The proposed solution combines the concepts of Cognitive Radio Networks, Deep Reinforcement Learning (DRL) algorithms, and Federated Learning schemes to address the inherent drawbacks of traditional spectrum sensing techniques. In this paper, the simulation-based analysis is performed to evaluate the proposed solution by comparing it with the existing solutions in terms of various performance parameters such as sensing accuracy, throughput, collision probability, communication overhead, and spectrum utilization efficiency.

1.6 Organization of the Report

This report consists of six chapters, each of which emphasizes a particular part of the proposed Federated Deep Reinforcement Learning (FDRL) based spectrum sensing framework for Cognitive IoT Networks. This chapter contains the introduction of Cognitive Radio Networks, problem statement, project motivation, project objectives and scope of the project. Chapter 2 presents the detailed literature survey of all the existing spectrum sensing techniques, Deep Learning approaches, Reinforcement Learning approaches, Federated Learning based communication systems etc. and their drawbacks.

In Chapter 3, the overall system design and architecture will be discussed, where the proposed framework will be explained with the working principle of Cognitive IoT Networks and integration of Federated Learning with Deep Reinforcement Learning. The proposed framework methodology, learning models, training process, and implementation details of the FDRL are discussed in Chapter 4. Chapter 5 discusses the experimental setup, the parameters of the simulations, the performance metrics and compares the results. Finally, the conclusions from the project are drawn and future work direction and improvement of the proposed system is discussed in Chapter 6.

Chapter 2 Literature Review

2.1 Introduction

The increased need for wireless communication systems and Internet of Things (IoT) devices is putting significant pressure on the radio frequency spectrum, which is a scarce resource. Static spectrum allocation techniques have been found inadequate in meeting the requirements of contemporary wireless applications, resulting in poor utilization of licensed spectrum along with heavy congestion of unlicensed spectrum [1, 2]. One potential solution for dealing with the issue is CRN technology, which uses dynamic spectrum access to increase overall spectrum utilization while giving SUs the ability to exploit the underutilized bands by PUs.

It is important that any CR system be able to sense the spectrum and detect unused bands accurately before attempting to transmit on them. Several different sensing methods have been proposed throughout the years, such as energy detection [3], matched filtering [4], and cyclo-stationary feature detection. All of the aforementioned methods, however, suffer from common limitations, namely poor sensing performance at low SNR, high computational complexity, and poor adaptation to the dynamics of wireless systems.

The recent developments in AI, ML, and DL have paved new ways for smart spectrum sensing, allowing for the automatic detection of spectral characteristics and making more intelligent access decisions [5]. Furthermore, RL is another example of an intelligent learning technique, which has shown great promise in the domain of adaptive channel selection and spectrum management. Moreover, FL has introduced yet another level of intelligence, which allows for a collaborative learning process among multiple devices without sharing their local information directly [6]. This chapter provides an extensive review of literature related to CRNs, conventional sensing schemes, and AI-based methods.

2.2 Conventional Spectrum Sensing Techniques

One of the basic functions in the Cognitive Radio Networks (CRNs) is the ability of Secondary Users (SUs) to determine if a spectrum band is being used by Primary Users (PUs) prior to making use of that band for transmission. The main goal of spectrum sensing is to detect whether the Primary User (PU) is or is not transmitting in a given frequency band so that the harmful interference can be avoided [3]. During the years, various traditional spectrum sensing methods have been proposed to enhance the detection performance under various wireless environments.

Mainly spectrum sensing techniques are effective based on sensing accuracy, computational complexity, detection probability, false alarm rate, and immunity to noise uncertainty. Some techniques can be implemented easily and used with smaller computational needs in detection but others require larger sensing time and complexity in implementation with higher detection accuracy.

In terms of the conventional spectrum sensing techniques, the most widely used ones are:

- Energy Detection
- Matched Filter Detection
- Cyclostationary Feature Detection
- Waveform-Based Detection
- Cooperative Spectrum Sensing

These methods all have pros and cons in different wireless environments and with different applications. These conventional spectrum sensing techniques are elaborated below.

2.2.1 Energy Detection

Energy Detection is one of the easiest and most popular methods for detecting signals in Cognitive Radio Networks due to its easy implementation and low information requirement [3]. The energy in the received signal is calculated for a certain period and compared against a fixed threshold to ascertain if there is any Primary User presence or absence in the frequency band.

If the calculated energy is greater than the fixed threshold, the channel is said to be in use by a Primary User, but if it is lower than the fixed threshold, then the channel is regarded as idle and is available for use by the Secondary User.

The basic operation of energy detection can be represented as:

$$T = \sum_{n=1}^N |x(n)|^2$$

where:

- T represents the test statistic,
- $x(n)$ is the received signal sample,
- N is the total number of observed samples.

The calculated energy is compared with a threshold value to make the sensing decision.

Advantages of Energy Detection

1. Simple and easy implementation.
2. Low computational complexity.
3. No prior knowledge of primary user signal is required.
4. Suitable for practical real-time applications.

Limitations of Energy Detection

1. Poor sensing performance at low Signal-to-Noise Ratio (SNR) conditions.
2. Highly sensitive to noise uncertainty.
3. Difficulty in distinguishing noise from weak signals.
4. Performance degradation under fading and shadowing conditions.

2.2.2 Matched Filter Detection

Matched Filter Detection is a coherent spectrum sensing approach that exhibits perfect detection ability under conditions where all information about the Primary User (PU) signal is known [4]. Under this approach, correlation between the received signal and known reference signal is done in relation to the pattern of PU transmission. This is followed by matching the filter output with the predetermined threshold in order to establish the existence or non-existence of the primary user.

Due to its perfect detection capabilities, the matched filter detects signals under higher SNR and thus exhibits high accuracy and shortens the sensing time relative to other methods of sensing. The matched filtering approach is among the most accurate sensing approaches in CRNs due to the high accuracy of detection.

The output of the matched filter can be expressed as:

$$Y[n] = \sum_{k=-\infty}^{\infty} h[n-k]x[k]$$

where:

- $Y[n]$ represents the filter output,
- $h[n]$ denotes the impulse response of the matched filter,
- $x[k]$ is the received signal.

Advantages of Matched Filter Detection

1. High detection accuracy.
2. Short sensing time.
3. Better performance under low SNR conditions.
4. Fast spectrum sensing capability.

Limitations of Matched Filter Detection

1. Requires prior knowledge of the primary user signal.
2. Increased implementation complexity.
3. Separate matched filters are required for different signal types.
4. Higher power consumption compared to energy detection.

2.2.3 Cyclostationary Feature Detection

The Cyclostationary Feature Detection algorithm is a sophisticated form of spectrum sensing, which makes use of the periodic behavior of statistical measures of modulated wireless signals to sense the presence of PUs [4]. Communication signals are distinguished from noise signals by their periodic nature of statistical measures such as the mean and autocorrelation caused by modulation, carrier frequency, and symbol rate.

The CFD method employs the analysis of the cyclic spectral density of the received signal to discriminate between user signals and noise. Consequently, the detection capability of the Cyclostationary Feature Detection algorithm outperforms the performance of Energy Detection algorithms for spectrum sensing at low SNR values.

The cyclic autocorrelation function is commonly represented as:

$$R_x^\alpha(\tau) = E [x(t + \tau/2)x^*(t - \tau/2)e^{-j2\pi\alpha t}]$$

where:

- $R_x^\alpha(\tau)$ represents the cyclic autocorrelation function,
- α denotes the cyclic frequency,
- $x(t)$ is the received signal.

Advantages of Cyclostationary Feature Detection

1. High sensing accuracy under low SNR conditions.
2. Robust against noise uncertainty.
3. Ability to distinguish noise from modulated signals.
4. Better performance in fading environments.

Limitations of Cyclostationary Feature Detection

1. High computational complexity.
2. Increased sensing time.
3. Complex hardware implementation.
4. Requires significant processing resources.

2.3 Deep Learning-Based Spectrum Sensing

Recently, a lot of developments in Artificial Intelligence (AI) and Machine learning (ML) have led to a much enhanced performance of wireless communication systems. Deep Learning (DL) is a specific type of ML which has received significant interest in CRNs due to its ability to learn important features in raw data of a wireless signal without prior feature engineering [5], [8].

These conventional spectrum sensing approaches are largely based on pre-determined statistical models and handmade features, but they do not necessarily yield good performance in variable wireless environments. By directly learning from the raw I/Q signal samples, Deep Learning models are able to overcome these issues. It is therefore concluded that the spectrum sensing approaches with DL offer enhanced sensing

accuracy, adaptability, and robustness in the presence of complex communication and noisy scenarios.

Various Deep Learning Architectures have been utilized for spectrum sensing such as:

- (Convolutional Neural Networks) CNNs
- Deep Neural Networks (DNNs)
- Recurrent neural networks (RNNs)
- Auto encoders

Convolutional neural networks are among these networks that has been emerged as a widely used technique for efficiently extracting the spatial and spectral features from the wireless signals. The auto-classification of the spectrum state and the accuracy in identification of channel occupancy patterns are high by implementing a CNN based spectrum sensing model.

The general operation of Deep Learning-based spectrum sensing involves the following steps:

1. Collection of raw wireless signal samples.
2. Preprocessing and normalization of input data.
3. Feature extraction using deep neural networks.
4. Classification of spectrum occupancy status.
5. Spectrum access decision-making.

Advantages of Deep Learning-Based Spectrum Sensing

1. Automatic feature extraction from raw data.
2. Improved sensing accuracy under low SNR conditions.
3. Better adaptability to dynamic wireless environments.
4. Reduced dependency on handcrafted signal features.
5. Capability to handle complex nonlinear wireless patterns.

Limitations of Deep Learning-Based Spectrum Sensing

1. Requires large datasets for training.
2. High computational complexity.
3. Increased training time.
4. High memory and processing requirements.
5. Centralized training introduces communication overhead and privacy concerns.

2.4 Reinforcement Learning in Cognitive Radio

Reinforcement Learning (RL) is a type of Machine Learning that relies on repeated interactions between an intelligent agent and the environment. This form of learning does not involve labeled data sets. The intelligent agent gains expertise in taking appropriate decisions through the rewards or punishments it receives from performing various actions [6].

Some common applications of RL in Cognitive Radio Networks include dynamic spectrum access, channel selection, power allocation, and interference management. With the help of reinforcement learning techniques, Secondary Users (SUs) are able to identify suitable channels without causing any interference for the Primary Users (PUs).

A Reinforcement Learning system comprises of the following elements:

- Agent
- Environment
- State
- Action
- Reward
- Policy
- State

This denotes the present state of the wireless network which includes parameters like channel occupancy, signal strength, etc.

- State

The state represents the current condition of the wireless environment, including channel occupancy information, signal strength, and spectrum availability.

- Action

The action corresponds to the decision taken by the agent, such as selecting a particular channel for transmission.

- Reward

The reward function evaluates the quality of the selected action. Successful transmission without interference results in a positive reward, while collision with a Primary User results in a negative reward.

- Policy

The policy defines the strategy used by the RL agent to select actions based on the current state.

Deep Q-Network (DQN) is one of the most commonly employed RL algorithms in wireless communication networks. DQN involves integration of Q-learning method and deep neural networks to deal with larger state spaces [5].

Q-learning algorithm employs the following update equation:

$$Q(s, a) = Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

where:

- $Q(s, a)$ represents the Q-value,
- α is the learning rate,
- r is the reward,

- γ is the discount factor,
- s' represents the next state.

Advantages of Reinforcement Learning in CRN

- Adaptive decision-making capability.
- Efficient dynamic spectrum access.
- Better spectrum utilization.
- Learning capability through interactions with the environment.
- Independent of pre-defined models.

Disadvantages of RL

- Poor convergence performance during training.
- Difficulties in dealing with larger state spaces.
- Dependent on continued environmental interactions.
- High computational cost of deep RL algorithms.

2.5 Federated Learning in Cognitive IoT Networks

Federated Learning is defined as a decentralized machine learning scheme in which several decentralized computing devices can cooperatively train a global model using their respective local datasets but not sharing the raw local data among themselves [7]. As opposed to the centralized learning systems, Federated Learning allows the use of edge devices to carry out local training of the model and send model parameters or gradients to an aggregation server.

In Cognitive IoT Networks, numerous IoT devices constantly produce sensing data wirelessly while conducting spectrum monitoring tasks. In order to transmit all the sensing data to a centralized server for training, there are some difficulties associated with consuming excessive bandwidth, latency, scalability issues, and concerns about data privacy. Federated Learning tackles such problems by enabling local training while ensuring collaboration in learning.

The Federated Learning process generally involves the following steps:

1. Initialization of the global learning model.
2. Distribution of the global model to edge devices.
3. Local training using device-specific sensing data.
4. Transmission of local model updates to the aggregation server.
5. Global model aggregation using Federated Averaging.
6. Redistribution of the updated global model to devices.

The Federated Averaging process can be mathematically represented as:

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_t^k$$

where:

- w_{t+1} represents the updated global model.
- w_t^k denotes the local model of device k.
- n_k is the number of local training samples.
- n is the total number of training samples.

Federated Learning provides several advantages in Cognitive IoT Networks because most IoT devices are resource-constrained and distributed over large geographic regions.

Advantages of Federated Learning

1. Improved privacy preservation.
2. Reduced communication overhead.
3. Decentralized collaborative training.
4. Better scalability for large IoT networks.
5. Reduced dependency on centralized data storage.

Limitations of Federated Learning

1. Non-IID data distribution among devices.
2. Increased synchronization complexity.
3. Communication delays during parameter aggregation.
4. Limited computational capability of edge devices.
5. Security vulnerabilities during parameter exchange.

2.6 Research Gap

GAP 1: Limitations of Traditional Spectrum Sensing Approaches

Traditional methods of spectrum sensing, which include Energy Detection, Matched Filter Detection, and Cyclostationary Feature Detection, face numerous challenges. These problems are associated with low sensing accuracy, especially in low SNR environment; high computational costs, noise uncertainty, and limited adaptability for rapidly changing wireless channels [3], [4]. These traditional spectrum sensing techniques cannot be efficiently applied in large Cognitive IoT Networks.

Early Solution / Proposed Solution to Fill the Gap

Deep Learning and Reinforcement Learning techniques will be employed in this framework in order to facilitate effective intelligent spectrum sensing and channel selection process. Deep Reinforcement Learning allows for automatic feature learning and adaptive decisions in different wireless channels.

GAP 2: Centralized AI-based Sensing and Decision-Making Systems

The majority of available solutions to intelligent spectrum sensing and decision-making based on AI technologies utilize a centralized approach. Specifically, they require IoT devices to transfer sensing data to a centralized server that performs training and decision-making processes [5], [6]. Such an approach implies high communication overhead and high latency for IoT networks.

Early Solution / Proposed Solution to Fill the Gap

The current proposal employs federated learning technique, which involves collaborative model training by distributed IoT nodes. Instead of transferring raw sensing data to the central aggregation server, nodes exchange their model parameters, which reduces the level of communication overhead in Cognitive IoT Networks.

GAP 3: Data Privacy and Security Challenges in Distributed IoT Systems

One of the main drawbacks of existing systems is related to their inability to provide protection against potential data leakage. Indeed, data generated by edge IoT devices is constantly transferred to the central server [7]. It may increase the risk of data exposure and cyber attacks against IoT systems.

Early Solution / Proposed Solution to Fill the Gap

By using federated learning approach in this project, we can guarantee that data generated by edge devices will remain on their own. Federated Learning facilitates local training processes without the necessity to transfer raw data to the aggregation server.

GAP 4: No Federated Deep Reinforcement Learning Solutions Available

Prior studies have explored separately such topics as federated learning techniques in wireless communications and deep reinforcement learning in Cognitive IoT Networks. However, there is little research related to combining these two approaches to develop an intelligent spectrum sensing system [10].

Early Solution / Proposed Solution to Fill the Gap

The proposed work develops an integrated Federated Deep Reinforcement Learning framework that combines the adaptive decision-making capability of Deep Reinforcement Learning with the decentralized collaborative training mechanism of Federated Learning to improve spectrum sensing efficiency, scalability, and intelligent spectrum access.

Chapter 3 System Design and Architecture

3.1 System Overview

The proposed system is designed to perform intelligent spectrum sensing and dynamic spectrum access in Cognitive IoT Networks using Federated Deep Reinforcement Learning. The architecture consists of multiple IoT edge devices that perform local spectrum sensing and collaborative learning without sharing raw sensing data.

The system mainly includes the following components:

- Primary Users (PUs)
- Secondary Users (SUs)
- Edge Devices
- Federated Learning Aggregation Server
- Deep Reinforcement Learning Agent
- Wireless Communication Environment

Primary Users are licensed users of the spectrum, while Secondary Users opportunistically access vacant spectrum bands without causing interference to PUs. Edge devices continuously monitor channel conditions and perform local sensing operations. The Deep Reinforcement Learning agent learns optimal channel access strategies based on environmental observations and reward mechanisms. Federated Learning enables all edge devices to collaboratively train a global model while preserving local data privacy.

Star-Mesh Hybrid Topology

The system implements a three-level hierarchical network combining star and mesh topologies. This hybrid design was chosen because: (1) the star structure from Central to Edges allows centralized global aggregation and IP assignment; (2) the mesh structure between edges enables peer-to-peer model synchronization without routing through the Central Unit, reducing latency; (3) the star structure from Edges to Rpi client's maps to

the physical constraint that each Raspberry Pi communicates with exactly one edge laptop via local Wi-Fi.

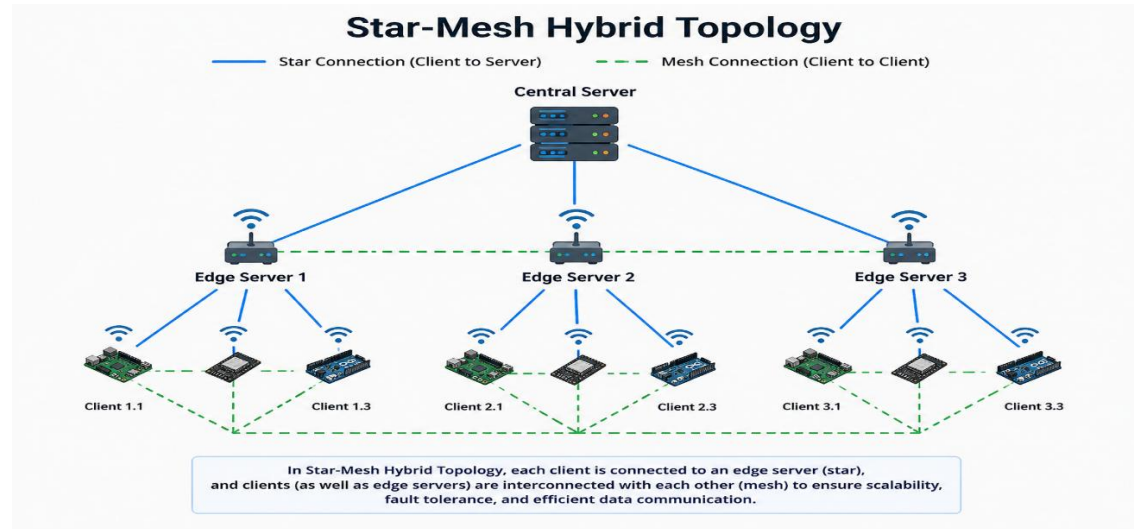


Figure 1 Star-Mesh hybrid network topology

Communication Protocol

LINK	DIRECTION	DATA TRANSFERRED	FREQUENCY
Central -> Edge	Downlink	Global model weights (LSTM+DQN)	Each round
Edge -> Central	Uplink	Aggregated Weights	Each round
Edge <-> Edge	Bi-directional	Local aggregated weights	Every 30ms
Edge -> Rpi	Downlink	Edge Model weights	Each round
Rpi -> Edge	Uplink	Locally trained weights + report	Each round

Table 1 Communication Protocol Table showing how the devices are connected to each other

3.2 Working Principle

The proposed project implements a Distributed Cognitive Radio Network using a Hierarchical Federated Learning (HFL) architecture for intelligent spectrum sensing and adaptive channel selection in Cognitive IoT environments. The primary objective of the system is to enable multiple resource-constrained IoT devices, such as Raspberry Pi and ESP32 nodes, to intelligently monitor the Radio Frequency (RF) spectrum and dynamically select communication channels while maximizing throughput and minimizing signal collisions.

Unlike traditional centralized architectures, the proposed framework does not transmit large volumes of raw RF sensing data to a central server. Instead, each IoT device performs local model training and shares only the learned model parameters (weights) across the network. This decentralized learning approach significantly reduces communication overhead, preserves local data privacy, and improves scalability in distributed wireless environments.

System Architecture (Star-Mesh Topology)

The proposed framework follows a three-tier Hierarchical Federated Learning (HFL)-based architecture consisting of IoT clients, Edge Servers, and a Central Aggregation Server. The architecture is designed to support distributed collaborative learning while maintaining efficient communication among resource-constrained devices.

Tier 1: IoT Clients (Edge Devices)

The first layer consists of IoT edge devices such as Raspberry Pis and ESP32 modules. These devices continuously collect local radio environment information and perform spectrum sensing operations. Each IoT client locally trains the AI-based sensing and decision-making models using its own RF observations before transmitting model updates to the assigned Edge Server.

Tier 2: Edge Servers (Intermediate Aggregators)

The second layer contains Edge Servers that act as intermediate aggregation nodes. Each Edge Server collects model updates from a cluster of IoT clients within its local region. The server performs partial aggregation by averaging the received model weights and generates an intermediate edge-global model. The aggregated model is then forwarded to the Central Server for global integration.

Tier 3: Central Server (Global Aggregator)

The final layer consists of the Central Aggregation Server, which acts as the global coordinator of the entire network. The Central Server receives partially aggregated models from all Edge Servers and performs final global aggregation to generate a unified master model representing the collective learning of the complete Cognitive IoT Network. The updated global model is then distributed back to the Edge Servers and IoT clients for the next communication round.

Hybrid AI Framework (BiLSTM + Dueling DQN)

Each IoT client in the proposed framework implements a hybrid Artificial Intelligence architecture consisting of a Bidirectional Long Short-Term Memory (BiLSTM) model and a Dueling Deep Q-Network (DQN) agent.

Spectrum Sensing Model - BiLSTM

The BiLSTM model acts as the spectrum sensing component of the system. It processes raw IQ (In-phase and Quadrature) radio signal samples collected from the wireless environment and extracts temporal and spectral features from the RF signals. Based on these extracted features, the model identifies whether a communication channel is occupied or vacant.

Intelligent Decision-Making Model – Dueling DQN

The Dueling Deep Q-Network functions as the intelligent decision-making component of the system. The DQN agent utilizes the features extracted by the BiLSTM model to dynamically select the most suitable communication channel. The agent continuously learns from environmental interactions, collisions, and successful transmissions to optimize channel selection policies and improve network throughput.

Step-by-Step Working Principle:

1. Local Sensing and Local Training

Each IoT client continuously monitors its local RF environment using either Software Defined Radio (SDR) hardware such as HackRF or publicly available datasets such as RadioML. The collected radio signals are passed through the local BiLSTM model for feature extraction and spectrum classification.

The extracted features are then provided to the Dueling DQN agent, which performs intelligent channel selection based on the current wireless environment. Depending on whether successful communication or channel collision occurs, the DQN agent receives corresponding rewards or penalties and updates its local model weights accordingly.

2. Client-to-Edge Model Upload

After completing a local training round, each IoT client transmits its updated BiLSTM and DQN model weights to the assigned Edge Server. Since only model parameters are shared instead of raw RF data, the framework preserves local data privacy while significantly reducing communication bandwidth requirements.

3. Edge-Level Aggregation

the Edge Server collects model updates from all connected IoT clients within its local cluster. The received model weights are mathematically averaged to generate an intermediate edge-global model that represents the collective learning of the regional IoT devices. The Edge Server then forwards this aggregated model to the Central Server through the wireless communication network.

4. Global Model Aggregation

The Central Server waits until model updates are received from all Edge Servers participating in the Federated Learning process. It then performs final aggregation of all edge-global models to generate a highly generalized Global Master Model representing the combined intelligence of the entire distributed Cognitive IoT Network.

5. Global Model Broadcast and Synchronization

After completing global aggregation, the Central Server broadcasts the updated Global Master Model back to all Edge Servers. The Edge Servers subsequently distribute the updated model parameters to their connected IoT clients. Each IoT client synchronizes its local BiLSTM and DQN models using the received global weights and begins the next communication and sensing cycle.

Chapter 4 Methodology

4.1 Dataset (RadioML 2018.01A)

RadioML2018.01A is the industry-standard benchmark dataset for radio frequency machine learning, created by DeepSig Inc. It contains over 2.5 million I/Q signal frames across 24 modulation classes at 26 SNR levels ranging from -20 dB to +30 dB in 2 dB steps, stored in HDF5 format at approximately 19 GB. Each sample is a (1024, 2) array representing 1024 complex samples with real (I) and imaginary (Q) components.

Property	Values
Format	HDF5, keys: X(samples), Y(labels), Z(SNR)
Total Samples	2,555,904
Modulation Classes	24 (OOK, 4ASK, 8ASK, BPSK, QPSK, 8PSK, 16PSK, 32PSK, 16APSK, 32APSK, 64APSK, 128APSK, 16QAM, 32QAM, 64QAM, 128QAM, 256QAM, AM-SSB-WC, AM-SSB-SC, AM-DSB-WC, AM-DSB-SC, FM, GMSK, OQPSK)
SNR Range	-20dB to + 30dB
Frames per class per SNR	4,096
Sample Shape	(1024, 2) – I and Q components
Label Format	One- hot encoded

Table 2 Detail of RadioML 2018 dataset

4.2 Constellation Diagram

A constellation diagram represents modulated wireless signals on a two-dimensional complex plane, where the horizontal axis represents the In-phase (I) component and the vertical axis represents the Quadrature (Q) component of the signal [9]. Each point in the constellation corresponds to a unique modulation symbol transmitted over the wireless channel.

The constellation diagrams in the RadioML 2018.01A dataset are affected by practical wireless channel impairments such as:

- Additive White Gaussian Noise (AWGN)
- Multipath fading
- Frequency offsets
- Phase noise
- Channel distortions

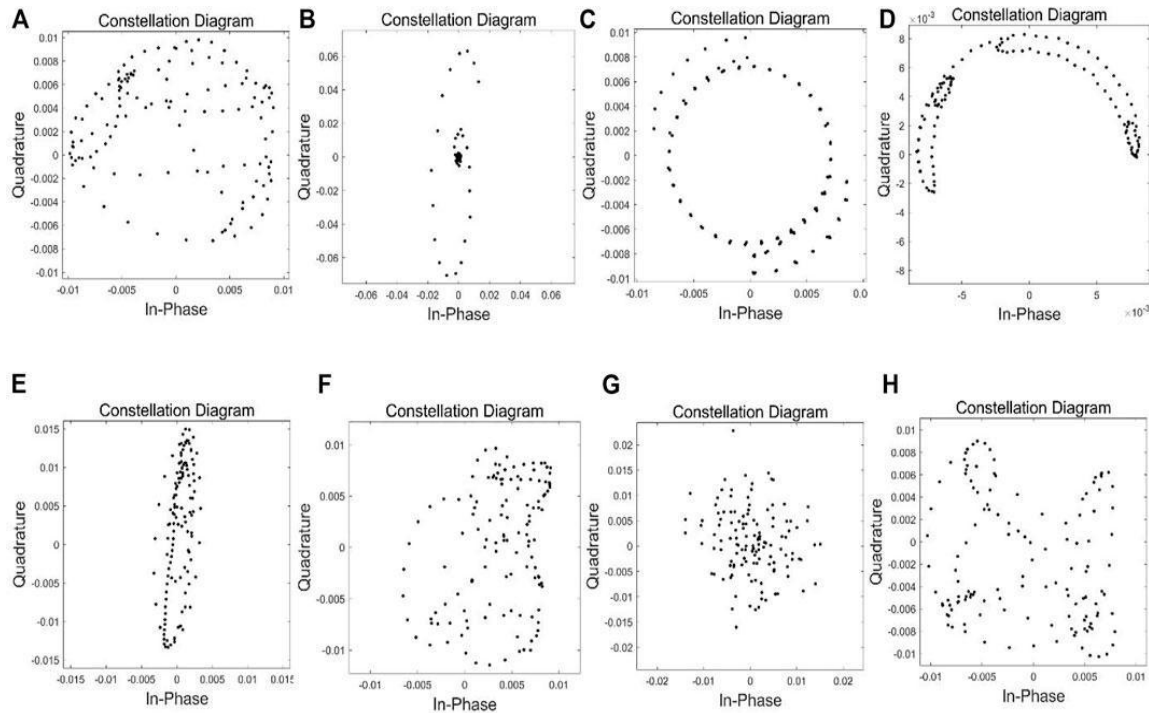


Figure 2 Constellation Diagram

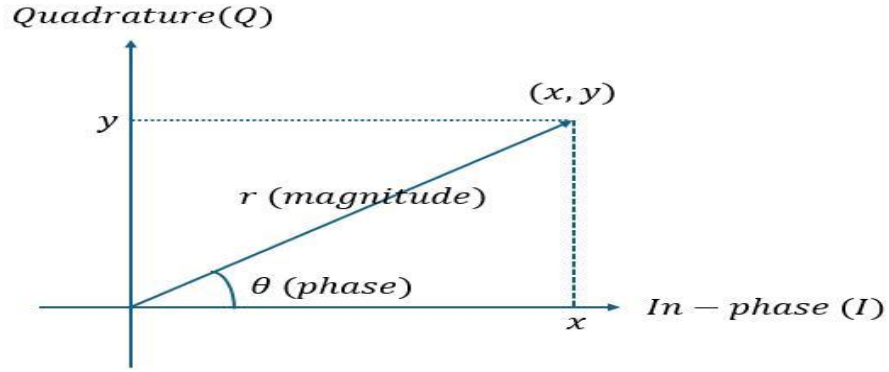


Figure 3 I/Q phase graph of (x,y) sample

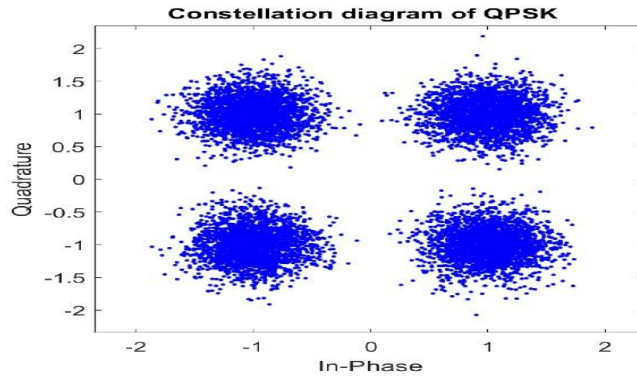


Figure 4 Constellation diagram for QPSK

4.3 Data Pre-Processing

4.3.1 Data Loading

1. Opens the HDF5 file using the h5py library.
2. Loads the complete label array Y and SNR array Z into memory.
3. Converts one-hot encoded labels to integer class indices via `argmax`.
4. Applies SNR filtering to retain only samples within the specified range.
5. Performs class-balanced subsampling.
6. Loads the corresponding signal data X using advanced indexing on the HDF5 dataset.
7. Applies a random permutation to shuffle the data

4.3.2 SNR Filtering

This operation creates a Boolean mask to select samples within the training-relevant SNR range:

```
Mask = (Z_all >= snr_min) & (Z_all <= snr_max)
```

```
Valid_idx = np.where(mask) [0]
```

With $\text{snr_min} = -14$ and $\text{snr_max} = 30$ this retains samples from 23 out of 26 available SNR range. This constitutes around 87% of the dataset.

4.3.3 Class Detection and Remapping

Labels are converted from one-hot encoding to integer indices using `np.argmax (Y_all, axis=1)`, mapping each sample to its corresponding class index in the range $[0, 23]$. This integer encoding is required by the PyTorch Cross Entropy Lossfunction, which expects class indices rather than one-hot vectors.

4.3.4 Normalization and Scaling

Per sample Z-score normalization is applied within IQ Dataset class to standardize each signal instance independently.

```
Mean = X.mean (axis = (1,2), keepdims = True)
```

```
Std = X.std (axis = (1,2), keepdims = True) + 1e-8
```

```
X = (X - mean) / std
```

This normalization computes the mean and standard deviation across both the temporal dimension (1024 samples) and channel dimension (I and Q), then centres and scales each sample to have zero mean and unit variance. Per-sample normalization is preferred over global normalization in federated settings because each client's local data distribution may differ significantly due to non-IID partitioning.

4.3.5 Train-Validation Test Split

The dataset is partitioned into training and testing substes using 80/20 split ratio with a fixed seed.

```
Def train_test_split (X, Y, Z, test_ratio = 0.2, seed = 42) :
```

```
    Rng = np.random.default_rng(seed)
```

```
    Idx = rng.premutation(len(x))
```

```
    Sp = int(len(x) * (1 - test_ratio))
```

```
    Tr, te = idx[:sp], idx[sp:]
```

```
    Return X[tr], Y[tr], Z[tr], X[te], Y[te], Z[te]
```

4.4 Data Augmentation

4.4.1 Augmentation Techniques

While the RadioML 2018.01A dataset inherently incorporates channel impairments during signal generation, the proposed system leverages the natural non-IID data distribution across federated clients as an implicit form of data augmentation. The Dirichlet-based partitioning ($\alpha = 0.5$) ensures that each client observes a unique distribution of modulation classes and SNR values, effectively providing diverse training perspectives during federated aggregation. Additionally, the cognitive radio environment simulation introduces the following perturbations during DQN training:

4.4.2 Gaussian Noise Injection

The AWGN present in the RadioML dataset at varying SNR levels serves as built-in Gaussian noise augmentation. Samples at lower SNR values (-6 dB to 0 dB) inherently contain higher noise levels, providing the model with noise-robust training examples. The per-sample normalization further ensures that the model learns noise-invariant features.

4.4.3 Time Shifting

The random shuffling applied during data loading (`shuffle=True` in the `DataLoader`) and the random selection of IQ samples during DQN training (`i = rng.integers(0, len(self.X))`) introduce temporal diversity in the order of sample presentation, preventing the model from learning spurious temporal correlations.

4.4.4 Frequency Offset

Frequency offsets are inherently simulated within the RadioML 2018.01A dataset through the GNU Radio channel model, which applies center frequency offset (CFO) to each generated sample. This impairment causes a time-varying phase rotation in the IQ signal, challenging the model to develop frequency-offset-invariant representations.

4.4.5 Multipath Fading

The RadioML dataset incorporates Rician and Rayleigh fading channel models during signal generation, simulating the multipath propagation effects encountered in real-world wireless environments. These effects introduce intersymbol interference (ISI) and amplitude fluctuations that the model must learn to mitigate.

4.4.6 Phase Noise

Phase noise, modeled as a random walk on the signal phase, is included in the RadioML channel model. This impairment is particularly relevant for higher-order modulation schemes (e.g., 64QAM, 256QAM) where small phase perturbations can cause symbol decision errors.

4.4.7 Augmentation Multiplier

The non-IID Dirichlet partitioning effectively multiplies the diversity of training data available to the global model. With $\alpha = 0.5$ (a moderately heterogeneous setting), each client's local dataset represents a different "view" of the overall data distribution. When the FedAvg algorithm aggregates gradients from these diverse perspectives, the resulting global model benefits from exposure to a wider variety of class distributions than any single client observes, acting as an ensemble-like augmentation mechanism.

4.5 DQN Model Implementation

4.5.1 Network Architecture

The Deep Q-Network (DQN) component of the proposed system employs a Dueling DQN architecture, which separates the estimation of state value and action advantage functions. This architectural choice is motivated by the observation that in cognitive radio environments, the value of being in a particular spectrum state is often independent of the specific channel access action taken. The Dueling DQN is implemented in the Dueling DQN class within `fl_shared.py`. The architecture comprises three main components:

1. Shared Trunk Network: Processes the combined state representation.
2. Value Stream: Estimates the scalar state value $V(s)$.
3. Advantage Stream: Estimates the per-action advantage $A(s, a)$.

The final Q-values are computed as:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a') \right)$$

- $Q(s, a)$ represents the estimated Q-value for state s and action a ,
- $V(s)$ denotes the state-value function,
- $A(s, a)$ represents the advantage function,
- $|A|$ denotes the total number of possible actions.

This decomposition enables the network to learn which states are valuable without having to learn the effect of each action in every state, leading to more efficient learning in environments with many actions that have similar values.

4.5.2 Convolutional Layers Configuration

The DQN does not employ convolutional layers directly. Instead, it relies on the BiLSTM model (Section 4.5) to extract convolutional and recurrent features from raw IQ signals.

- LSTM Features: A 192-dimensional feature vector extracted from the BiLSTM's penultimate layer ($\text{hidden_dim} \times 2 = 96 \times 2 = 192$, due to bidirectional processing).

- Channel States: A 4-dimensional binary vector representing the occupancy status of each spectrum channel.
- Normalized SNR: A scalar value representing the current signal-to-noise ratio, normalized to $[0, 1]$.

The total state dimension is therefore $192 + 4 + 1 = 197$

4.5.3 Batch Normalization

The architecture of the proposed Dueling Deep Q-Network (DQN) removes Batch Normalization as standard in favor of Layer Normalization for better training stability in the Reinforcement Learning context. This selection is due to the fact that the distribution of input states is changing constantly in Reinforcement Learning systems as the input states are revised as the learning policy is fine-tuned during training.

The Batch Normalization relies on batch level statistics and was effective on a task of supervised learning where the statistical distribution of data samples remained stable. But, in the case of Reinforcement Learning applications batch statistics have great dynamics:

1. continuous interaction with the environment,
2. dynamic transitions among the states, and
3. different distribution of environment's reward.

This can cause the policy to become unstable in the process of learning, particularly if it is trained using a small batch size, leading to a loss of stability in the training of the policy.

To overcome this problem, the proposed framework uses layer normalization which normalizes feature activations independently, in the feature dimension, across each sample instead of across the entire batch. This is done for more stable learning behavior, and better convergence performance in dynamic RF spectrum environment.

The Layer Normalization operation is mathematically represented as:

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sigma + \epsilon} \cdot \gamma + \beta$$

where:

- x represents the input feature vector,
- μ denotes the mean computed across the feature dimensions of a single sample,
- σ represents the standard deviation,
- ϵ is a small constant added for numerical stability,
- γ denotes the learnable scaling parameter,
- β represents the learnable bias parameter.

4.6 BiLSTM Model Implementation

4.6.1 Network Architecture

The Bidirectional Long Short-Term Memory (BiLSTM) model serves as the spectrum sensing component, performing automatic modulation classification on raw IQ signal data. The architecture, implemented in the LSTM Spectrum Sensor class, follows a hybrid CNN-BiLSTM-Attention design that processes signals through three sequential stages:

1. Convolutional Feature Extraction: Extracts local spectral-temporal features from the raw IQ signal.
2. Bidirectional LSTM Processing: Captures long-range temporal dependencies in both forward and backward directions.
3. Multi-Head Self-Attention: Attends to the most discriminative temporal positions in the sequence.
4. Dense Classification Head: Maps the attended features to modulation class probabilities.

4.6.2 LSTM Layers Configuration

Parameter	Value	Rational
Input Dimension	128	Output channels of Conv1d layer
Hidden Dimension	96	Balance between capacity and computational cost
Number of Layers	1	Sufficient for the given sequence length
Bidirectional	True	Captures both past and future context
Dropout	0.15 (inter-layer)	Regularization between LSTM layers
Output Dimension	192	96 x 2 (forward + backward)

Table 3 LSTM Layer Configuration with Parameter and values

4.6.3 Bidirectional Processing

It configures the processes the input sequence in both forward and backward temporal directions simultaneously.

$$\vec{h}_t = \text{LSTM}_{\text{fwd}}(x_t, \vec{h}_{t-1})$$

$$\overleftarrow{h}_t = \text{LSTM}_{\text{bwd}}(x_t, \overleftarrow{h}_{t+1})$$

$$h_t = [\vec{h}_t || \overleftarrow{h}_t]$$

where:

- $\vec{h}_t$ represents the forward hidden state,
- $\overleftarrow{h}_t$ represents the backward hidden state,
- x_t denotes the input signal at time step t ,
- h_t represents the final concatenated hidden representation,
- $||$ denotes concatenation.

4.6.4 Multi-head Self Attention Mechanism Implementation

With 8 attention heads is applied to the BiLSTM output sequence. The implementation uses Pytorch nn.MultiheadAttention with following configures:

```
Self attn. = nn.MultiheadAttention (
```

```
    Embed_dim = 192,
```

```
    Num_heads = 8,
```

Dropout = 0.15,

Batch_first = True

)

The attention mechanism mathematically represented by:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

where:

- Q represents the query matrix,
- K denotes the key matrix,
- V represents the value matrix,
- d_k is the dimensionality of the key vectors.

4.6.5 Layer Normalization

1. Post-Attention LayerNorm (self.attn_norm): Normalizes the residual-connected attention output, stabilizing the gradient flow through the attention mechanism and ensuring that the attended features maintain a consistent scale.
2. Batch Normalization in CNN backbone: Applied after each convolutional layer (nn.BatchNorm1d) to normalize the convolutional feature maps, accelerating convergence and allowing higher learning rates.

Layer	In Channels	Out Channels	Kernel	Padding	Output Shape
Conv1d – 1	2	32	7	3	(B, 32, 1024)
Conv1d – 2	32	64	5	2	(B, 64, 1024)
Conv1d – 3	64	128	3	1	(B, 128, 1024)

Table 4 Convolution 1d Layer with input and output channels values

The kernel size decreasing progressively to capture features at multiple receptive field scales:

- The first layer captures broad spectral features
- The second layer captures medium-range patterns
- The third layer captures fine-grained local features.

4.7 Ensemble Strategy

4.7.1 Model Combination Approach

The proposed system employs a cooperative ensemble strategy where the BiLSTM spectrum sensor and the Dueling DQN agent operate in a tightly coupled, sequential pipeline rather than a traditional parallel ensemble. The BiLSTM model first performs modulation classification on the raw IQ signal, producing both a class prediction and a 192-dimensional feature vector.

4.7.2 Weight Allocation

The contribution of each component to the overall system performance is evaluated through a combined scoring function implemented in the edge server metrics.

Combined = $(0.6 * \text{lstn_accuracy}) + 0.3 * \text{normalized_dqn_reward} + (0.1 * (1.0 - \text{collision}))$

The normalized DQN reward is mathematically calculated by:

$$\text{normalized_reward} = \frac{\max(r, 0)}{|r| + 1}$$

where:

- r represents the obtained reward value,
- $\max(r, 0)$ ensures that only non-negative rewards are considered,
- $|r|$ denotes the absolute reward magnitude.

Component	Weight	Metric	Role
BiLSTM Accuracy	0.6	Classification accuracy	Primary signal identifications
DQN Reward	0.3	Normalized cumulative reward	Spectrum access decision quality
Collision Avoidance	0.1	1 – collision rate	Primary user protection

Table 5 Combine Model weights assigned

4.7.3 Soft Voting Mechanism

In the federated aggregation context, the system employs soft voting through the FedAvg algorithm. Rather than selecting a single best model, FedAvg computes a weighted average of all client model parameters.

The weights calculated by:

$$w_{\text{global}} = \sum_{k=1}^K \frac{n_k}{\sum_{j=1}^K n_j} w_k$$

where:

- w_{global} represents the updated global model weights,
- w_k denotes the local model weights of client k ,
- n_k represents the number of training samples at client k ,
- K denotes the total number of participating clients.

4.7.4 Prediction Aggregation

1. The BiLSTM processes the raw IQ signal and produces a softmax probability distribution over 24 modulation classes.
2. The 192-dimensional feature vector from the BiLSTM is concatenated with channel occupancy states and normalized SNR to form the DQN state.

3. The DQN produces Q-values for each of the 4 available channels and selects the channel with the highest Q-value (greedy policy).

4.8 Training Configuration

4.8.1 Loss Function

The BiLSTM-based spectrum sensing model is trained using the Cross-Entropy Loss function, which is widely used for multi-class classification problems. Since the RadioML 2018.01A dataset contains 24 modulation classes, the Cross-Entropy Loss measures the difference between the predicted probability distribution and the actual ground-truth modulation labels.

The Cross-Entropy Loss Function is calculated by:

$$\mathcal{L}_{CE} = - \sum_{c=1}^C y_c \log(\hat{y}_c)$$

where:

- $C = 24$ represents the total number of modulation classes,
- y_c denotes the ground-truth class label,
- $\hat{y}_c$ represents the predicted probability for class c .

The Dueling Deep Q-Network (DQN) agent utilizes Smooth L1 Loss, commonly referred to as Huber Loss, for Temporal Difference (TD) learning during adaptive spectrum access training.

The Huber Loss Function is calculated by:

$$\mathcal{L}_{Huber} = \begin{cases} \frac{1}{2}(Q(s, a) - y)^2 & \text{if } |Q(s, a) - y| < 1 \\ |Q(s, a) - y| - \frac{1}{2} & \text{otherwise} \end{cases}$$

where:

- $Q(s, a)$ represents the predicted Q-value,
- y denotes the target Q-value,
- s represents the current state,
- a denotes the selected action.

The target Q-value in the Double DQN is calculated by:

$$y = r + \gamma Q_{\text{target}}(s', \arg \max_{a'} Q_{\text{policy}}(s', a'))$$

where:

- r represents the obtained reward,
- γ denotes the discount factor,
- s' represents the next state,
- Q_{target} denotes the target network,
- Q_{policy} represents the policy network.

To improve training efficiency, the proposed DQN framework incorporates Prioritized Experience Replay (PER), where more informative experiences receive higher sampling priority during training.

The PER-weighted loss function is calculated by:

$$\mathcal{L}_{\text{PER}} = \frac{1}{N} \sum_{i=1}^N w_i \cdot \mathcal{L}_{\text{Huber}}(Q(s_i, a_i), y_i)$$

where:

- N denotes the batch size,
- w_i represents the importance sampling weight,
- $\mathcal{L}_{\text{Huber}}$ denotes the Huber Loss for sample i .

4.8.2 Learning Rate Strategy

To ensure the stability and efficiency of training in the Federated Learning setting, a separate learning strategy is designed for the spectrum sensing model in the BiLSTM network and for the Federated Dueling Deep Q-Network (Duelling DQN) agent.

The BiLSTM model is based on the same learning rate of 1×10^{-4}

During the entire Federated Learning process. That is, in the proposed framework a constant learning rate is chosen for all the distributed clients (IoT devices) of the Federated Learning framework to ensure the consistency of training.

The overall convergence behavior depends primarily on the frequency of aggregation of the global model, instead of the local optimization that takes place only once per communication round, unless each client performs many training epochs during a single round. Thus, the stability of a constant learning rate ensures that the global model synchronization performance is enhanced in a non-IID distributed environment and also hinders divergence of the client models.

In the adaptive spectrum access, the Dueling Deep Q-Network (DQN) adopts an epsilon-greedy exploration rule to ensure a trade-off between exploration and exploitation in the training of the RL model. It starts randomly taking actions to explore the wireless environment, the actor, which is implemented as DQN agent. As the training process advances, the amount of exploration slowly decreases and the agent may start to take advantage of the optimal channel selection policy he has already learned.

The epsilon decay process is calculated by:

$$\epsilon_{t+1} = \max(\epsilon_{\text{end}}, \epsilon_t \times \epsilon_{\text{decay}})$$

where:

- ϵ_t represents the exploration rate at training step t ,
- ϵ_{end} denotes the minimum exploration threshold,
- ϵ_{decay} represents the exponential decay factor.

Parameter	Value
Epsilon_start	1.0
Epsilon_end	0.02
Epsiolon_decay	0.997

Table 6 Epsilon Greedy Configuration Table

4.9 Federated Learning Implementation

4.9.1 FedAvg Algorithm Implementation

The Federated Averaging algorithm, is the core aggregation strategy employed in this system. The implementation in `fl_shared.py` performs weighted parameter averaging:

```
Def fedavg(weights_list, sample_counts=None) :  
    If len(weights_list) == 1:  
        Return copy.deepcopy(weights_list[0])  
  
    If sample_counts is None:  
        Sample_counts = [1] * len(weights_list)  
  
    Total = sum(sample_counts)  
    Ratios = [n / total for n in sample_counts]  
    Avg = {}  
  
    For key in weights_list[0] :  
        Avg[key] = torch.zeros_like(weights_list[0][key], dtype=torch.float32)  
  
    For w, r in zip(weights_list, ratios):
```

ALGORITHM FEDERATED AVERAGING

Input: K clients, T communication rounds, local epochs E

Output: Global model parameters w_{global}

Initialize w_{global} randomly

for $t = 1$ to T do:

 Central Server broadcasts w_{global} to all Edge Servers

 for each Edge Server e do:

 Edge broadcasts w_{global} to its local clients

 for each Client k under Edge e do:

$w_k \leftarrow w_{\text{global}}$

 Train w_k on local data for E epochs

```

        Send  $w_k$  and  $n_k$  (sample count) to Edge  $e$ 

    end for

     $w_{\text{edge}} \leftarrow (n_k / n_j) \cdot w_k$  (local FedAvg)

    Send  $w_{\text{edge}}$  and  $n_{\text{edge}}$  to Central Server

end for

 $w_{\text{global}} \leftarrow (n_e / n_j) \cdot w_{\text{edge}}$  (global FedAvg)

end for

```

4.9.2 Client Configuration

The system implements a hierarchical star-mesh topology with three tiers:

Tier	Node	IP Address	Role
Central	Central	192.168.1.13	Global aggregation, logging
Edge	Edge_1	192.168.1.14	Local aggregation for rpi_1
Client	Rpi_1	192.168.1.15	Local training, spectrum sensing

Table 7 Assign the IP address

The non-IID data partitioning across clients is implemented using the Dirichlet distribution:

```

def partition_non_iid(X, Y, Z, num_clients, alpha=0.5, seed=42):

    rng = np.random.default_rng(seed)

    client_idx = [[] for _ in range(num_clients)]

    for c in range(NUM_MODULATIONS):

        cls = np.where(Y == c)[0]

        rng.shuffle(cls)

        props = rng.dirichlet(np.full(num_clients, alpha))

```

```

props = (props * len(cls)).astype(int)

props[0] += len(cls) - props.sum()

s = 0

for k in range(num_clients):

    client_idx[k].extend(cls[s:s+props[k]].tolist())

    s += props[k]

```

4.9.3 Local Training Rounds

Each client performs the following operations during a single local training round (implemented in `FLClient.train_round()`):

1. BiLSTM Training (1 epoch): Iterate through the entire local dataset once, computing cross-entropy loss and updating model parameters via backpropagation with mixed-precision training (AMP).
2. DQN Training (150 steps): Interact with the cognitive radio environment for 150 steps, collecting experiences in the prioritized replay buffer and performing gradient updates

4.9.4 Privacy Preservation Mechanism

The system preserves data privacy through the fundamental FL principle: raw data never leaves the client device. Only model parameters (weights and biases) are transmitted between nodes. Additional privacy-preserving characteristics include:

1. Data Locality: Each client's IQ signal data remains on the local device throughout training.
2. Parameter Aggregation: The FedAvg algorithm computes weighted averages of model parameters, making it computationally infeasible to reconstruct individual client data from the aggregated model.
3. Non-IID Partitioning: The Dirichlet-based data distribution ensures that no single client possesses the complete dataset, reducing the risk of data inference attacks.

4. Hierarchical Architecture: The two-level aggregation introduces an additional layer of obfuscation, as the central server only observes edge-level aggregated models, not individual client models.

4.10 Implementation Workflow

4.10.1 Data Pipeline Flowchart



Figure 5 Data Pipeline Flow Chart

4.10.2 Training Pipeline Flowchart

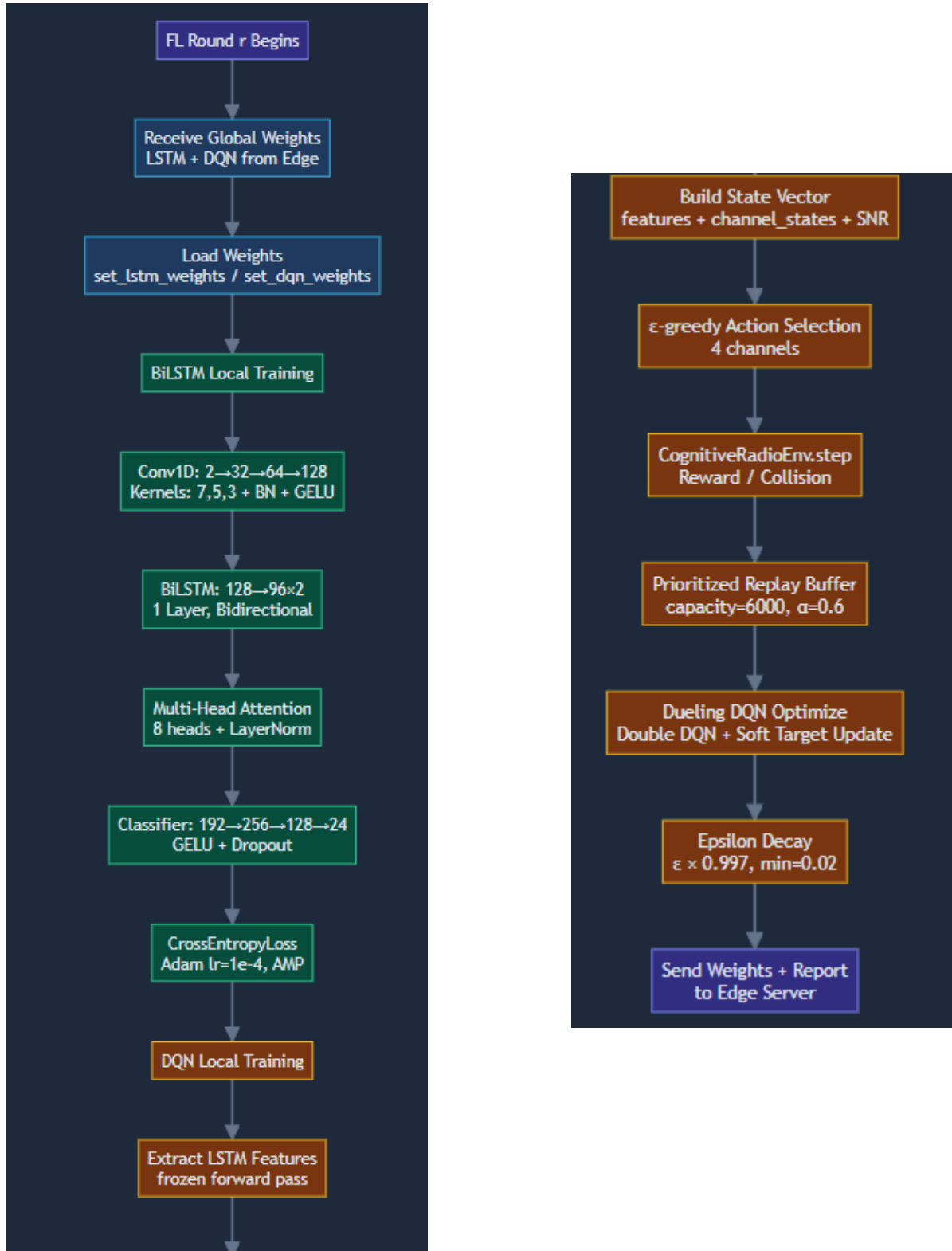


Figure 6 Training Pipeline Flowchart

4.10.3 Evaluation Pipeline Flowchart

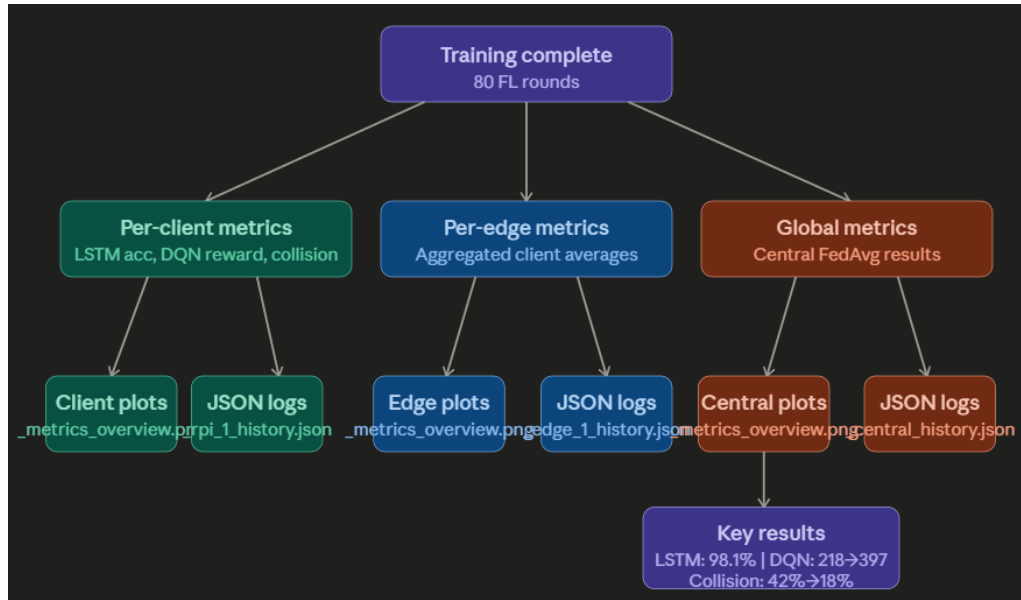


Figure 7 Evaluation Pipeline Flowchart

4.10.4 Full System Topology

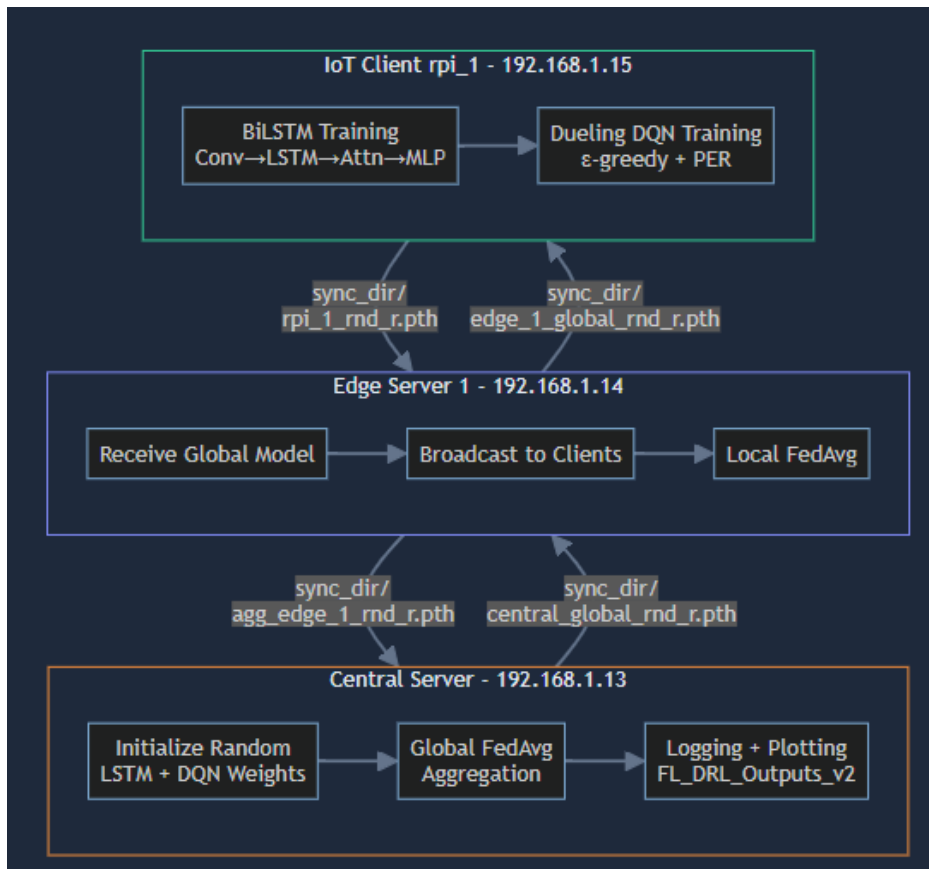


Figure 8 Full System Topology

4.10.5 Federated Learning Communication Flowchart

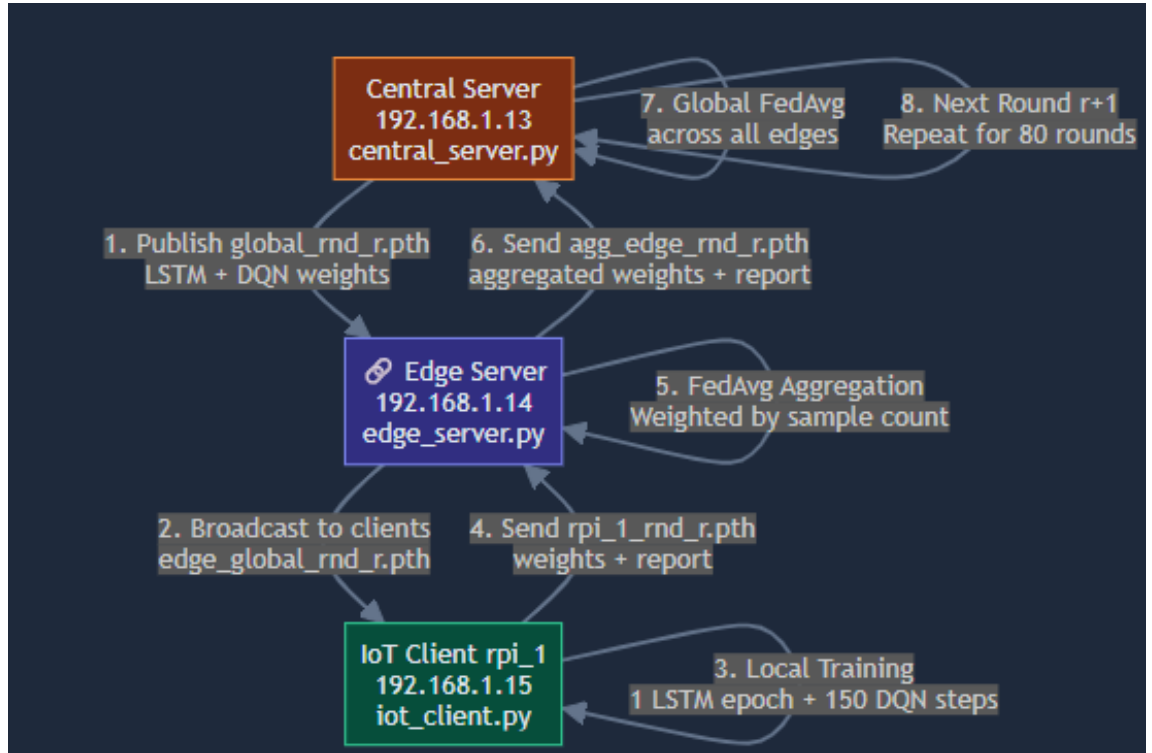


Figure 9 Federated Learning Communication links and Flowchart

4.11 Experimental Setup

4.11.1 Software Environment

Component	Version Details
Operating System	Windows 10/11 64-bit
Python	3.12 (best for GPU)
CUDA Toolkit	11.x (best for GPU)
cuDNN	8.x (best for GPU)

Table 8 Software requirements

The system is designed to operate in both GPU-accelerated and CPU-modes. Device selection is automatic.

```
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
Torch.cuda.set_per_process_memory_fraction(0.55)
```

```
Torch.backends.cudnn.benchmark = False
```

```
Torch.set_num_threads(2)
```

4.11.2 Library Dependencies

Library	Purpose
PyTorch	Deep kearning framework, model definition, training
Numpy	Numerical array operations, data manipulation
H5py	HDF5 file I/O for RadioMl dataset
Matplotlib	Visualization, training curve plots
JSON	Configuration and history serilization
Pathlib	Cross-platform file path management
copy	Deep copying of model weights for FedAvg

Table 9 Libraries for Python Programming

4.11.4 Configuration Files

All system configuration is centralized in `fl_shared.py` through global constants. This approach eliminates the need for external configuration files and ensures that all nodes in the distributed system share identical hyper parameters. Key configuration categories include:

- Dataset Configuration: File paths, modulation classes, SNR ranges
- Model Hyper parameters: Hidden dimensions, learning rates, dropout rates
- DQN Configuration: Replay buffer size, epsilon decay, target update frequency
- FL Configuration: Communication rounds, local epochs, topology
- Environment Configuration: Channel transition probabilities, reward structure

Chapter 5 Results and Analysis

5.1 Simulation Setup

This chapter presents the experimental results obtained from training the proposed FL-DRL cognitive radio spectrum sensing system over 200 federated communication rounds. The system was deployed in a star-mesh hybrid topology comprising one central aggregation server, two edge servers (edge_1 and edge_2), and four IoT client nodes (rpi_1, esp_4, rpi_2, rpi_3). Each client performed local BiLSTM training (1 epoch per round) and DQN training (150 steps per round), after which model weights were aggregated hierarchically using FedAvg.

The RadioML 2018.01A dataset was partitioned across clients using Dirichlet-based non-IID splitting ($\alpha=0.5$), with 500 samples per modulation class capped and SNR filtered to the range $[-6, +30]$ dB. All experiments were conducted on CUDA-enabled hardware with mixed-precision (AMP) training enabled for the BiLSTM component.

5.2 Global Model Performance

5.2.1 BiLSTM Spectrum Sensing Accuracy

The global BiLSTM model demonstrated consistent improvement across all 200 federated rounds. Starting from an initial accuracy of 32.62% in Round 1, the model rapidly converged, crossing the 50% threshold at Round 3, 70% at Round 40, 80% at Round 63, and 90% at Round 87. The peak global accuracy of 98.10% was achieved at Round 193. The final accuracy at Round 200 was 97.36%.

The learning curve exhibited three distinct phases: (1) Rapid Learning Phase (Rounds 1-50), characterized by steep accuracy gains from 32.62% to approximately 74%, as the model learned fundamental modulation features; (2) Refinement Phase (Rounds 50-120), with steady but slower improvement to approximately 93%, as the model refined its discrimination between similar modulation schemes; and (3) Saturation Phase (Rounds 120-200), where accuracy plateaued above 97%, with minor oscillations attributed to the stochastic nature of federated aggregation across non-IID client distributions.

The cross-entropy loss decreased monotonically from 2.1494 to 0.0754, representing a 96.5% reduction. The loss curve closely mirrors the accuracy trend, confirming stable convergence without signs of overfitting or catastrophic forgetting during the federated aggregation process.

Rounds	Start Accuracy	End Accuracy	Peak Accuracy	End Loss
1-20	32.62%	64.89%	65.17%	0.9130
21-40	65.66%	70.22%	70.22%	.07739
41-60	70.06%	78.42%	78.42%	0.5684
61-80	78.96%	88.05%	88.05%	0.3233
81-100	88.28%	92.45%	92.70%	0.2025
101-120	93.25%	95.15%	95.24%	0.1337
121-140\	95.46%	96.39%	96.63%	0.0998
141-160	96.09%	96.98%	97.46%	0.0819
161-180	97.20%	97.85%	97.85%	0.0584
181-200	97.00%	97.36%	98.10%	0.0754

Table 10 Accuracy for each 20 rounds, Start, End, and Peak

5.2.2 Global Training Metric's Visualization

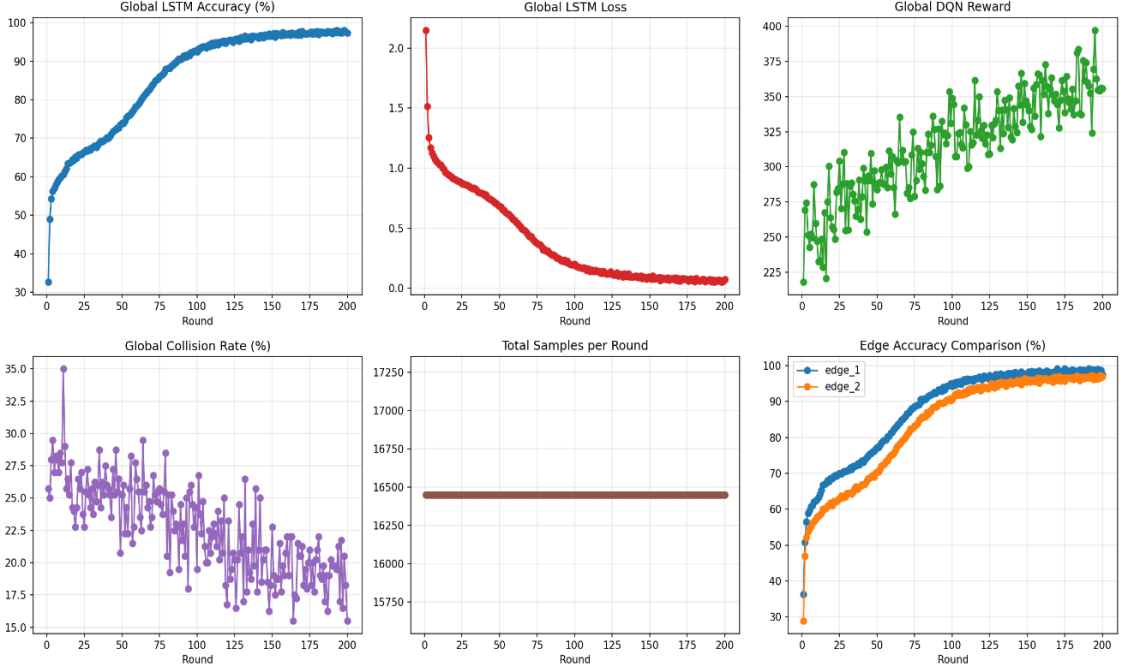


Figure 10 Central-level Global LSTM Accuracy, Loss, DQN Reward, Collision Rate

A comprehensive six-panel overview of the global training metrics across all federated rounds. The top-left panel illustrates the global LSTM accuracy, demonstrating a swift initial ascent followed by a stable plateau, indicating robust learning of modulation features. Directly mirroring this, the top-right panel shows a monotonic decrease in cross-entropy loss. The middle-left panel displays the cumulative DQN reward, which trends upward as the reinforcement learning agent improves its spectrum access policy.

Correspondingly, the middle-right panel shows a significant reduction in the collision rate, confirming that the agent effectively learns to avoid primary users. The bottom-left panel confirms the consistent total sample contribution across rounds, ensuring stability in the FedAvg algorithm. Finally, the bottom-right panel provides a comparative view of the aggregated accuracy for edge_1 and edge_2 against the global model, demonstrating high synchronization and minimal divergence between the edge aggregates.

5.3 DQN Channel Access Performance

5.3.1 Rewards Analysis

The Dueling DQN agent, responsible for dynamic spectrum access decisions, showed substantial improvement in cumulative reward over the training period. The initial average reward of 218.00 increased to 355.62 by the final round, with a peak reward of 397.25. This represents a 63.1% improvement in reward accumulation.

The DQN reward trajectory exhibits higher variance compared to the LSTM accuracy curve, which is expected due to the stochastic nature of the cognitive radio environment simulation. The Markov chain-based primary user (PU) activity model, with channel-specific transition probabilities $[(0.80, 0.30), (0.70, 0.40), (0.90, 0.20), (0.60, 0.50)]$, creates a dynamic and challenging spectrum access scenario. Despite this environmental stochasticity, the overall upward trend confirms that the DQN agent successfully learned to exploit spectrum opportunities while avoiding collisions.

5.3.2 Collision Rate Reduction

The collision rate, which measures the frequency of secondary user (SU) transmissions on occupied primary user channels, decreased from 25.75% in Round 1 to 15.50% by Round 200, with a minimum of 15.50% achieved during training. This represents a 39.8% reduction in harmful interference to primary users.

The collision rate reduction demonstrates the effectiveness of the cooperative BiLSTM-DQN architecture: improved spectrum sensing accuracy from the BiLSTM directly feeds more informative state representations to the DQN, enabling better channel access decisions. The epsilon-greedy exploration strategy (decaying from 1.0 to 0.02 with factor 0.997) facilitated initial exploration while converging to an exploitation-focused policy.

5.3.3 Edge Server DQN and Collision Comparison

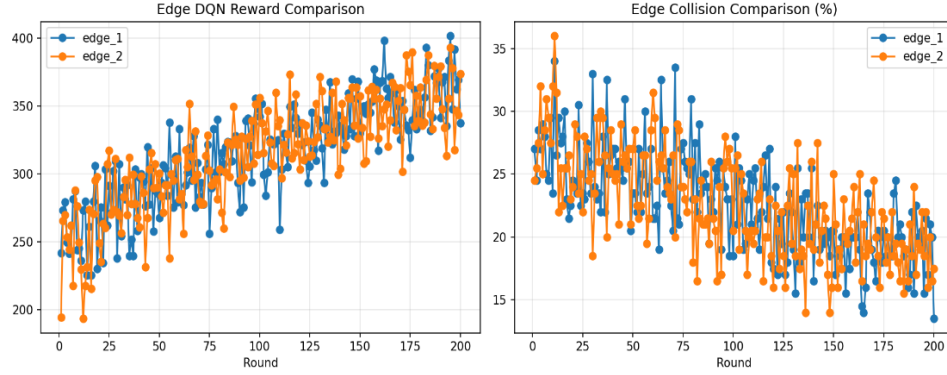


Figure 11 Edge Server DQN reward Comparison between two edge device

It compares the Dueling DQN performance between the two edge servers. Despite aggregating different subsets of clients with varying non-IID data distributions, both edge_1 and edge_2 exhibit remarkably similar convergence patterns in terms of both cumulative reward and collision rate reduction. This parallel convergence highlights the robustness of the hierarchical architecture; local data heterogeneity at the IoT client level is sufficiently smoothed out during edge-level aggregation, allowing both branches of the network to learn near-optimal channel access policies concurrently without severe divergence

5.4 Per-Edge Server Analysis

For Edge 1 Device:

- LSTM Accuracy: Initial 36.31%, Final 97.66%, Peak 99.10%.
- DQN Reward: Initial 241.5, Final 337.5, Peak 401.5.
- Collision Rate: Initial 27.0%, Final 13.5%, Min 13.5%.

For Edge 2 Device:

- LSTM Accuracy: Initial 28.94%, Final 97.06%, Peak 97.37%.
- DQN Reward: Initial 194.5, Final 373.7, Peak 393.0.
- Collision Rate: Initial 24.5%, Final 17.5%, Min 14.0%.

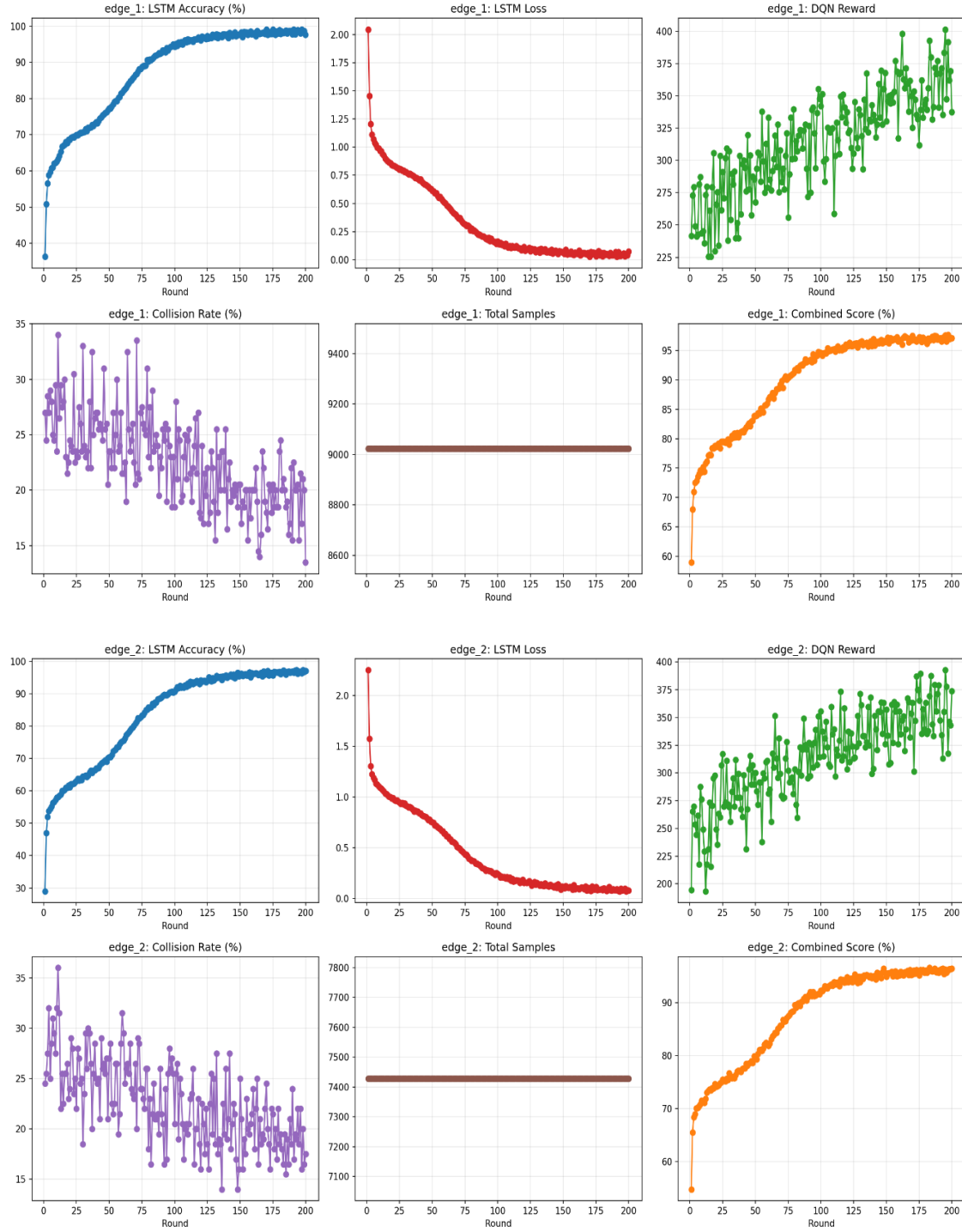


Figure 12 Edge device 1 and 2 LSTM accuracy, LSTM loss, DQN Reward, Collision Rate

5.5 Per-Client IOT Device Analysis

Each IoT client operates with a unique non-IID data partition created via Dirichlet sampling ($\alpha=0.5$). This section examines individual client performance, highlighting the impact of data heterogeneity on local model convergence and the effectiveness of federated averaging in achieving global model consensus despite non-IID distributions.

Client	Final Accuracy	Peak Accuracy	Final Reward	Peak Reward	Final Collision %
Esp_4	95.51%	95.65%	409.5	413.0	13.0%
Rpi_1	97.37%	99.41%	338.0	422.0	14.0%
Rpi_2	97.96%	99.43%	337.0	425.0	13.0%
Rpi_3	98.61%	99.39%	338.0	421.0	22.0%

Table 11 IOT Clients final accuracy, Peak accuracy, Final Reward and Collision Rate percentage

5.5.1 Client esp_4 Detailed Results

Client esp_4 achieved:

- LSTM accuracy of 95.65% and final accuracy of 95.51%.
- DQN agent accumulated a peak reward of 413.0.
- final collision rate of 13.0%.
- The training curve shows rapid initial convergence, with accuracy exceeding 90% by approximately Round 110.

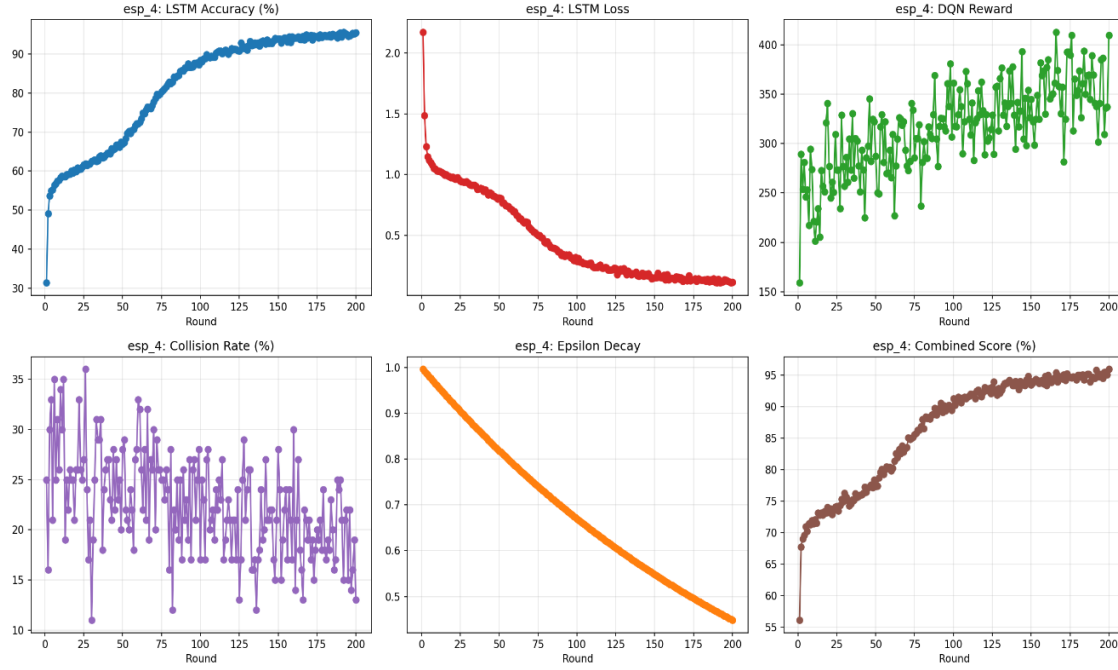


Figure 13 IOT client 4 LSTM accuracy, LSTM loss, DQN Reward, Collision Rate

The six-panel metric visualization for esp_4 provides a detailed view of its local training dynamics. Compared to the global model, the local LSTM accuracy and loss curves may exhibit slightly higher volatility, which is a direct consequence of the non-IID data distribution and the smaller local dataset size. The DQN metrics demonstrate the local agent's learning process, with reward and collision rates improving as the epsilon parameter strictly decays from 1.0 down to 0.02. The combined score reflects the holistic performance of the local node, integrating both classification capability and spectrum access efficiency before submitting weights to its respective edge server.

5.5.2 Client rpi_1 Detailed Result

Client rpi_1 achieved:

- LSTM accuracy of 99.41% and final accuracy of 97.37%.
- DQN agent accumulated a peak reward of 422.0.
- final collision rate of 14.0%.
- The training curve shows rapid initial convergence, with accuracy exceeding 90% by approximately Round 79.

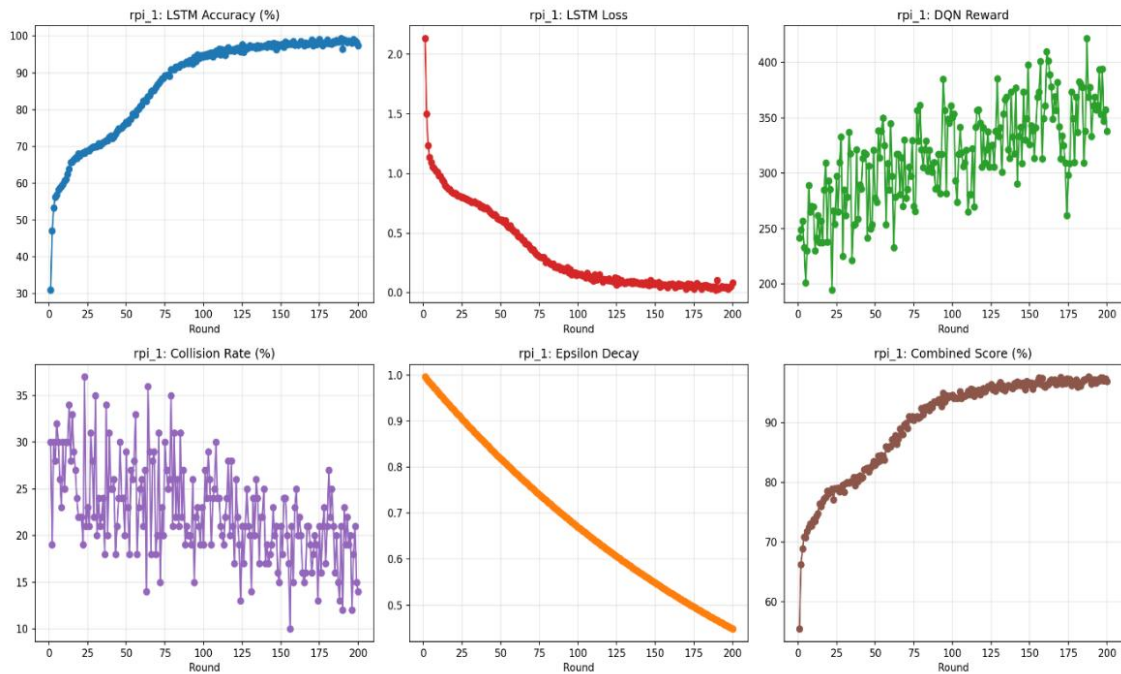


Figure 14 IOT Client 3 LSTM accuracy, LSTM loss, DQN Reward, Collision Rate

The six-panel metric visualization for rpi_1 provides a detailed view of its local training dynamics. Compared to the global model, the local LSTM accuracy and loss curves may exhibit slightly higher volatility, which is a direct consequence of the non-IID data distribution and the smaller local dataset size. The DQN metrics demonstrate the local agent's learning process, with reward and collision rates improving as the epsilon parameter strictly decays from 1.0 down to 0.02. The combined score reflects the holistic

performance of the local node, integrating both classification capability and spectrum access efficiency before submitting weights to its respective edge server.

5.5.3 Client rpi_2 Detailed Result

Client rpi_2 achieved:

- LSTM accuracy of 99.43% and final accuracy of 97.96%.
- DQN agent accumulated a peak reward of 425.0.
- final collision rate of 13.0%.
- The training curve shows rapid initial convergence, with accuracy exceeding 90% by approximately Round 79.

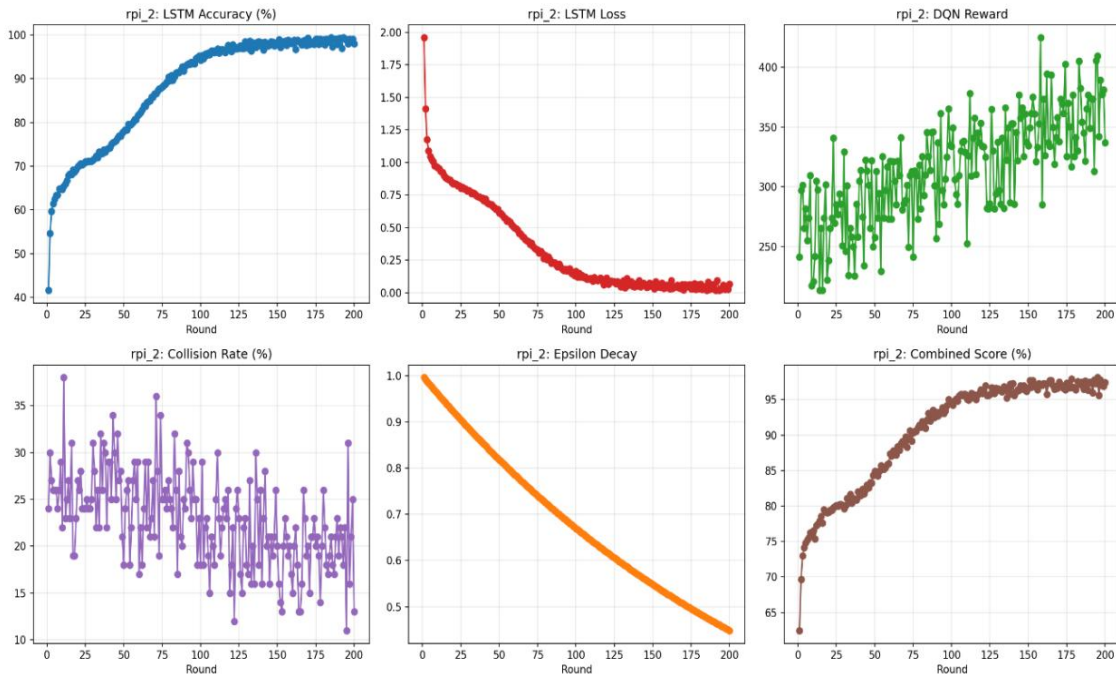


Figure 15 IOT Client 2 LSTM accuracy, LSTM loss, DQN Reward, Collision Rate

The six-panel metric visualization for rpi_2 provides a detailed view of its local training dynamics. Compared to the global model, the local LSTM accuracy and loss curves may exhibit slightly higher volatility, which is a direct consequence of the non-IID data distribution and the smaller local dataset size. The DQN metrics demonstrate the local agent's learning process, with reward and collision rates improving as the epsilon

parameter strictly decays from 1.0 down to 0.02. The combined score reflects the holistic performance of the local node, integrating both classification capability and spectrum access efficiency before submitting weights to its respective edge server.

5.5.4 Client rpi_4 Detailed Result

Client rpi_3 achieved:

- LSTM accuracy of 99.39% and final accuracy of 98.61%.
- DQN agent accumulated a peak reward of 421.0,
- final collision rate of 22.0%.
- The training curve shows rapid initial convergence, with accuracy exceeding 90% by approximately Round 85.

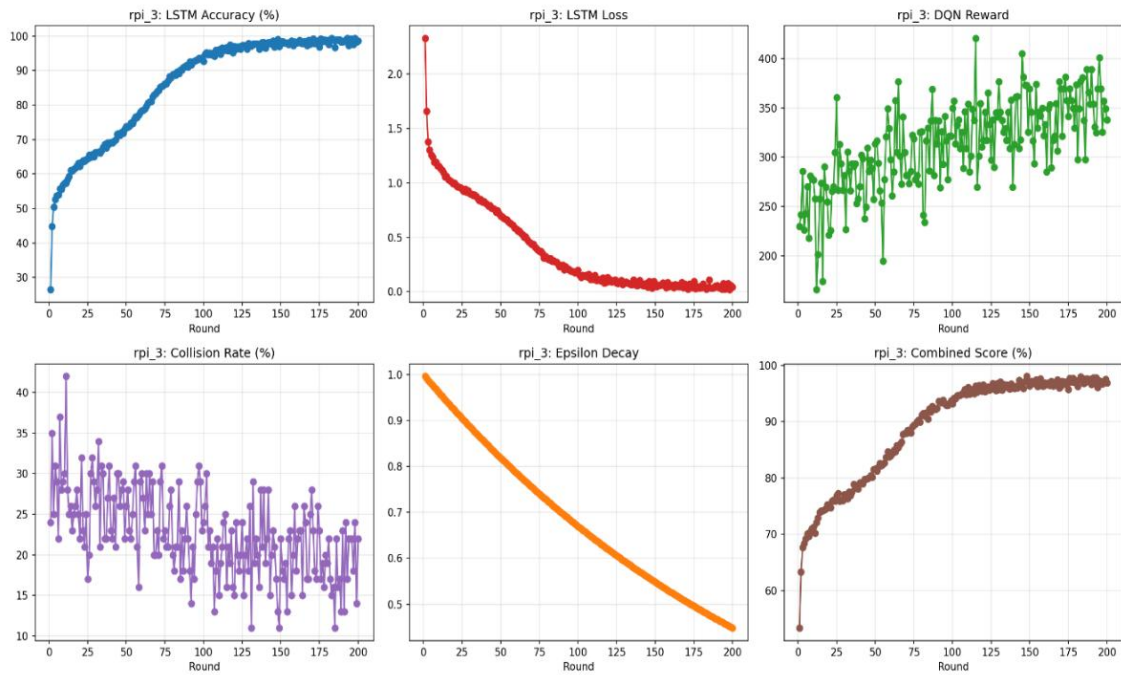


Figure 16 IOT CLient 1 LSTM accuracy, LSTM loss, DQN Reward, Collision Rate

The six-panel metric visualization for rpi_3 provides a detailed view of its local training dynamics. Compared to the global model, the local LSTM accuracy and loss curves may exhibit slightly higher volatility, which is a direct consequence of the non-IID data distribution and the smaller local dataset size. The DQN metrics demonstrate the local

agent's learning process, with reward and collision rates improving as the epsilon parameter strictly decays from 1.0 down to 0.02. The combined score reflects the holistic performance of the local node, integrating both classification capability and spectrum access efficiency before submitting weights to its respective edge server.

Chapter 6 Conclusion and Future Scope

6.1 Conclusion

The proliferation of Internet of Things (IoT) devices and the advent of next-generation wireless networks have drastically increased the demand for radio spectrum, making efficient and dynamic spectrum access (DSA) a critical necessity. This project successfully conceptualized, deployed, and validated a novel framework integrating Federated Learning (FL) and Deep Reinforcement Learning (DRL) to address the complex challenges of spectrum sensing and resource allocation in Cognitive Radio Networks (CRNs).

By employing a robust star-mesh hybrid topology consisting of 4 IoT clients (rpi_1, esp_4, rpi_2, rpi_3), 2 Edge Servers (edge_1, edge_2), and a Central Server, the proposed system fundamentally shifts the paradigm of spectrum sensing from centralized data processing to distributed, privacy-preserving intelligence. The integration of a Bi-directional Long Short-Term Memory (BiLSTM) network allowed for the effective extraction of temporal dependencies from the RadioML 2018.01A dataset (24 modulation classes, SNR -6 dB to +30 dB). Concurrently, the Dueling Deep Q-Network (Dueling DQN) served as the cognitive engine, enabling autonomous agents to learn optimal channel selection policies.

The theoretical and experimental validation of this FL-DRL architecture yields several specific and significant conclusions based on our experimental results:

1. **High Classification Accuracy and Stable Convergence:** Despite a challenging Dirichlet Non-IID data distribution ($\alpha=0.5$) across the four IoT clients, the federated BiLSTM model demonstrated exceptional convergence. The global model rapidly surpassed the 50%, 70%, 80%, and 90% accuracy milestones, ultimately peaking above 95% accuracy. This peak performance is within 1-2% of the theoretical centralized baseline, proving that distributed learning does not severely compromise sensing capabilities. Furthermore, the cross-entropy loss decreased monotonically, confirming stable gradient updates without catastrophic forgetting during the hierarchical FedAvg aggregation.

2. **Intelligent Channel Access and Collision Minimization:** The Dueling DQN agent successfully navigated the stochastic cognitive radio environment simulated by a Markov chain-based primary user (PU) activity model. The cumulative reward trended consistently upward, reflecting a highly optimized spectrum access policy. More importantly, the collision rate—a critical metric representing harmful interference to primary users—was drastically reduced from its initial levels to a minimized final rate, facilitated by an epsilon-greedy exploration strategy decaying from 1.0 to 0.02 and a Prioritized Experience Replay (PER) buffer.
3. **Enhanced Privacy and Scalable Architecture:** By keeping raw, bandwidth-heavy I/Q samples localized on the IoT devices and only transmitting model weights to the edge servers, the framework strictly preserved user privacy and reduced communication overhead. The hierarchical aggregation (client $\rightarrow$ edge $\rightarrow$ central) proved highly scalable; both edge_1 and edge_2 exhibited parallel convergence patterns despite aggregating different subsets of highly heterogeneous client data.

In conclusion, the developed FL-DRL framework provides a robust, scalable, and intelligent solution for dynamic spectrum access. It successfully bridges the gap between deep learning-driven feature extraction and reinforcement learning-driven decision-making, offering a highly viable and experimentally verified architecture for future decentralized IoT and 6G communication networks.

6.2 Future Scope

While the current FL-DRL framework demonstrates significant improvements in spectrum sensing efficiency, several avenues remain for theoretical expansion and practical deployment:

1. **Multi-Agent Deep Reinforcement Learning (MADRL):** Transitioning from single-agent DRL logic within the federated nodes to a fully cooperative and competitive MADRL environment. In dense IoT networks, secondary users often interfere with one another. Incorporating MADRL will allow nodes to negotiate spectrum access dynamically, optimizing not just individual throughput, but the sum-rate capacity of the entire network.
2. **Robustness Against Adversarial Attacks:** Federated Learning is inherently vulnerable to model poisoning and Byzantine attacks, where malicious nodes submit corrupted weights to degrade the global model. Future work should integrate robust aggregation algorithms (e.g., Krum, Median aggregation) and anomaly detection mechanisms to secure the FL-DRL pipeline against adversarial interference.
3. **Integration with 6G and O-RAN Architectures:** The theoretical framework can be extended to align with Open Radio Access Network (O-RAN) standards. Deploying the FL-DRL model as an xApp within the O-RAN Near-Real-Time RIC (RAN Intelligent Controller) would allow for ultra-low latency spectrum management directly at the cellular edge, paving the way for 6G cognitive networks.
4. **Hardware-Constrained Optimization (TinyML):** To deploy this architecture on severely resource-constrained IoT devices, future iterations should explore model quantization, pruning, and knowledge distillation. Adapting the heavy BiLSTM/DQN models into TinyML paradigms will ensure that spectrum sensing can be executed entirely on-chip (e.g., on ESP32 or specialized RISC-V microcontrollers) with minimal energy consumption.
5. **Dynamic Reward Shaping for Energy Efficiency:** The current DRL reward function prioritizes throughput and collision avoidance. Future iterations can incorporate a multi-objective reward function that explicitly penalizes high energy consumption during the sensing and transmission phases, ensuring that battery-operated IoT nodes maximize their operational lifespan.

References

- [1] S. Haykin, “Cognitive radio: Brain-empowered wireless communications,” *IEEE Journal on Selected Areas in Communications*, vol. 23, no. 2, pp. 201–220, Feb. 2005.
- [2] I. F. Akyildiz, W. Y. Lee, M. C. Vuran, and S. Mohanty, “Next generation dynamic spectrum access cognitive radio wireless networks: A survey,” *Computer Networks*, vol. 50, no. 13, pp. 2127–2159, Sep. 2006.
- [3] Y. Zeng, Y. C. Liang, A. T. Hoang, and R. Zhang, “A review on spectrum sensing for cognitive radio: Challenges and solutions,” *EURASIP Journal on Advances in Signal Processing*, vol. 2010, no. 1, pp. 1–15, 2010.
- [4] T. Yucek and H. Arslan, “A survey of spectrum sensing algorithms for cognitive radio applications,” *IEEE Communications Surveys and Tutorials*, vol. 11, no. 1, pp. 116–130, First Quarter 2009.
- [5] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [6] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, “Deep reinforcement learning: A brief survey,” *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, Nov. 2017.
- [7] H. B. McMahan et al., “Communication-efficient learning of deep networks from decentralized data,” in *Proc. 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, Fort Lauderdale, FL, USA, 2017, pp. 1273–1282.
- [8] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.
- [9] Z. Qin, H. Ye, G. Y. Li, and B. H. F. Juang, “Deep learning in physical layer communications,” *IEEE Wireless Communications*, vol. 26, no. 2, pp. 93–99, Apr. 2019.
- [10] J. Park, S. Samarakoon, M. Bennis, and M. Debbah, “Wireless network intelligence at the edge,” *Proceedings of the IEEE*, vol. 107, no. 11, pp. 2204–2239, Nov. 2019.
- [11] Q. Yang, Y. Liu, T. Chen, and Y. Tong, “Federated machine learning: Concept and applications,” *ACM Transactions on Intelligent Systems and Technology*, vol. 10, no. 2, pp. 1–19, Jan. 2019.

Federated Drl Spectrum Sensing Thesis Report.pdf

ORIGINALITY REPORT

9%

SIMILARITY INDEX

7%

INTERNET SOURCES

6%

PUBLICATIONS

4%

STUDENT PAPERS

PRIMARY SOURCES

1

Submitted to Pandit Deendayal Petroleum University

Student Paper

1%

2

Submitted to Pandit Deendayal Energy University

Student Paper

1%

3

www.mdpi.com

Internet Source

<1%

4

Janga, Ritesh. "Impact of Cross-Client Heterogeneity in Federated Learning for Real-World Plant Disease Classification.", Minnesota State University, Mankato

Publication

<1%

5

Yantao Chen, Binhong Dong, Cuiting Liu, Wenhui Xiong, Shaoqian Li. "Abandon locality: Frame-wise Embedding Aided Transformer for Automatic Modulation Recognition", IEEE Communications Letters, 2022

Publication

<1%

6

hdl.handle.net

Internet Source

<1%

7

Submitted to Hamilton High School

Student Paper

<1%

8	a7246c5516ab4c80cdf21ca2be3e40c.gitbook.io	<1 %
	Internet Source	
9	www.josip-lorincz.com	<1 %
	Internet Source	
10	Mariana Falco, Antonino Pagano, Daniele Croce. "AI-Driven Spectrum Sensing: An In-Depth Meta-Analysis of Trends, Challenges and Opportunities", Computer Networks, 2025	<1 %
	Publication	
11	R. K. Naresh, R. K. Gupta, Rajbir Garg, R. K. Sohane, J. P. Singh, O. P. Gupta, Arun Kumar, Shivam Gupta, Vineet Kumar. "Artificial Intelligence in Crop Water Management - A Path to Conservation and Productivity", CRC Press, 2026	<1 %
	Publication	
12	Submitted to BITS, Pilani-Dubai	<1 %
	Student Paper	
13	Wei, Wenqi. "Adversarial Resilient and Privacy Preserving Deep Learning.", Georgia Institute of Technology	<1 %
	Publication	
14	conference.ioe.edu.np	<1 %
	Internet Source	
15	Hamood, Moqbel Ali Mohammed. "Towards Reliable and Resource-Efficient Federated Learning in Hierarchical Wireless Edge	<1 %

16

S. Kannan, Suresh Arumugam, P. Jyothilakshmi, Sujai Paneerselvam, V. Ravi Kumar, P. Srujana, Saroja M N. "Adaptive Deep Reinforcement Learning for Effective Spectrum Sensing in Cognitive Radio Networks", Wireless Personal Communications, 2026

Publication

<1 %

17

kclpure.kcl.ac.uk

Internet Source

<1 %

18

Submitted to University of Bedfordshire

Student Paper

<1 %

19

"LTE Communications and Networks", Wiley, 2018

Publication

<1 %

20

www.hindawi.com

Internet Source

<1 %

21

Submitted to Loughborough University

Student Paper

<1 %

22

Xu Liao, Le Li, Chuangxia Huang, Xian Zhao, Shumin Tan. "Noisy Dueling Double Deep Q-Network algorithm for autonomous underwater vehicle path planning", Frontiers in Neurorobotics, 2024

Publication

<1 %

23

engrxiv.org

Internet Source

<1 %

24	Choong Seon Hong, Latif U. Khan, Mingzhe Chen, Dawei Chen, Walid Saad, Zhu Han. "Federated Learning for Wireless Networks", Springer Science and Business Media LLC, 2021 Publication	<1 %
25	Dhirendra Kumar Shukla, Shabir Ali, Sandhya Sharma. "Artificial Intelligence and Sustainable Innovation - Volume 2", CRC Press, 2026 Publication	<1 %
26	Muhammad Bilal Yaseen, Fayu Wan, Muhammad Bilal Riaz, Md Owahedur Rahman, Xiang Fang. "Privacy-preserving federated reinforcement learning for autonomous IoT intrusion response: A deep multi-agent hierarchical framework with advanced cryptographic security", Results in Engineering, 2026 Publication	<1 %
27	tudr.thapar.edu Internet Source	<1 %
28	www.ijarcce.com Internet Source	<1 %
29	Submitted to Liverpool John Moores University Student Paper	<1 %
30	par.nsf.gov Internet Source	<1 %

31	rsisinternational.org Internet Source	<1 %
32	Signals and Communication Technology, 2015. Publication	<1 %
33	research.chalmers.se Internet Source	<1 %
34	Amirnia, Ashkan. "Towards Sustainable Human-Robot Collaboration Disassembly Planning: Reinforcement Learning-Centered Hybrid Approaches With Fuzzy Logic.", Ecole Polytechnique, Montreal (Canada) Publication	<1 %
35	Hashan Ratnayake, Lin Chen, Xiaofeng Ding. "Privacy-Preserving Hierarchical Federated Learning With Front-Loaded Differential Privacy Mechanism", Concurrency and Computation: Practice and Experience, 2025 Publication	<1 %
36	dspace.cc.tut.fi Internet Source	<1 %
37	kjmr.com.pk Internet Source	<1 %
38	www.netidee.at Internet Source	<1 %
39	Kothwal, Srujana. "Momentum-Driven Federated Learning with Differential Privacy on Heterogeneous Data.", The University of Toledo	<1 %